

**"Good health is not something we can buy.
However, it can be an extremely
valuable savings account"**

— Anne Wilson Schaef

1 bit (sign bit) 

Max length no. supported will be of 64 bits. $2^{63}-1 \rightarrow \text{max}$

For 32 bits no. $-(2^{31}-1) \text{ to } (2^{31}-1)$ data type $\downarrow$ long long (LLONG-MAX)

$\text{INT-MAX} = 2^{31}-1$ $\downarrow$ int

$\text{INT-MIN} = -(2^{31}-1)$ $\downarrow$ int

$\text{LLONG-MIN} = -2^{63}$

Primitive data type

Data Type	Size	Description
int	32 bits	length of binary rep. is 32 (goes till 10^9)
float	32 bits	length of no. = 10
double	64 bits	length of no. = 20
boolean	8 bits / 1 byte	(goes till 10^{19})
char	8 bits / 1 byte	
long	64 bits	

1 bit for sign bit

- Functions like sort, lower-bound, upper-bound, binary search ~~needs~~ normal comparator (function based)
- You can pass normal func or functor. (class object as function)

For ex :- `sort(arr, arr+n, vector<int>, greater<int>());`

template <typename T>
class greater {
public:
bool operator()(T a, T b) { return a > b; }
};

For class based STL like sets, multisets, $\downarrow$ you need $\downarrow$ for comparison = greater

Things to Remember

- 1) (10/20) Array/Vector size limit $\rightarrow 10^6$
- $\Rightarrow$ Array of size 10^7 to 10^8 can be declared globally (on heap) $\rightarrow$ Dynamic memory
- $\Rightarrow$ 10^7 size array doesn't work in any function / int main due to smaller sized stack. $\rightarrow$ Static memory

$\Rightarrow$ At these times, you can use

`vector<vector<ll>> arr;` // declared globally

$\Rightarrow$ If there is a 2D array $N \times M$ then $N \times M \leq 10^7$ will be given in the problems.
 (will remain unless we delete it)

`arr = vector<vector<ll>>(n, vector<ll>(m));`

~~Inside function~~
 $\downarrow$ Inside function
 Reserving $n \times k \times m$ for each testcase

- 2) When you use "using namespace std" and declare a global variable named "count" or something which also is a func. then it will throw reference to "count" is ambiguous.

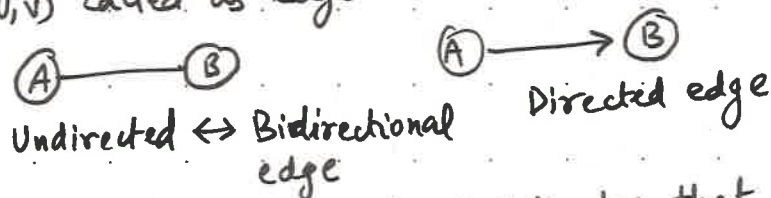
Rather use other variable names (good practice)

or remove "using namespace std"
 use "std::cout", "std::count" etc.

Graphs

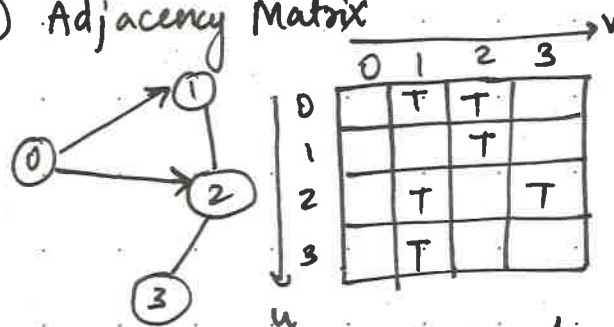
- Graph Representations

1. A finite set of vertices also called as nodes
2. A finite set of ordered pair of the form (u, v) called as edge.



The pair of the form (u, v) indicates that there is an edge from vertex u to vertex v .

1) Adjacency Matrix



Adv: Accessing an edge for removal or check takes $O(1)$ time i.e. $m[u][v] == T$?

Disadv: Consumes more space $O(V^2)$.
 Adding a vertex also takes $O(V^2)$
 To find neighbours $- O(V)$

② Edge list

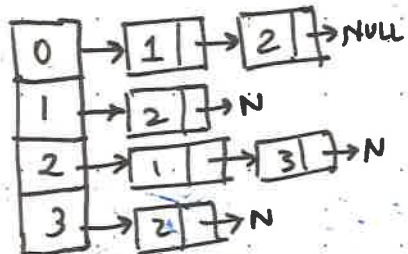
$V = \{0, 1, 2, 3\}$

$E = \{(0,1), (0,2), (1,2), (2,1), (2,3), (3,2)\}$

↳ stores all edges & vertices separately

② Adjacency List

An array of lists is used (list arrays).
Size of array = no. of vertices.



↑ key list<data-type>

Adv:- Saves space $O(V + |E|)$.
In worst case, there can be $(V, 2)$ no. of edges in a graph thus consuming $O(V^2)$ space.
Adding a vertex is easier.

Neighbour $\rightarrow O(\text{neighbours})$

Disadv:- Finding whether there is an edge from u to v is inefficient and takes $O(V)$.

④ Implicit Graph

0	0	1	1
1	1	0	1
1	0	0	1
0	0	0	1

Graph Adjacency List Implementation (Undirected or Bidirectional)

class Graph { // Array of list

int V; $\rightarrow$ list<int> *l;

public:

Graph(int V) {

this $\rightarrow$ V = V;

l = new list<int>[V]; }

void addEdge(int x, int y) {

l[x].push-back(y);

l[y].push-back(x); }

void print() {

for (int i = 0; i < V; i++) {

cout << "vertex " << i << " -> ";

for (int x : l[i]) {

cout << x << ", ";

cout << endl;

}; }

int main() {

Graph g(4);

g.addEdge(0, 1);

g.addEdge(0, 2);

g.addEdge(2, 3);

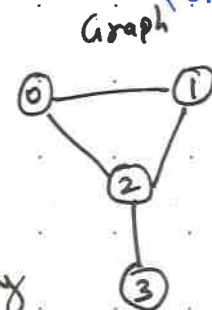
g.addEdge(1, 2); }

o/p:- Vertex 0 $\rightarrow$ 1, 2,

Vertex 1 $\rightarrow$ 0, 2,

Vertex 2 $\rightarrow$ 0, 3, 1,

Vertex 3 $\rightarrow$ 2,



Array of lists of size V.

0 $\rightarrow$ [1, 2] L1

1 $\rightarrow$ [2, 0] L2

2 $\rightarrow$ [3, 0, 1] L3

3 $\rightarrow$ [2] L4

Q. When we use "this $\rightarrow$ " in class?

When local variable passed in class function is same as ~~object~~ class' data member.

Ex:- V is data member of Graph and also a local variable in ~~main~~ Constructor.

Graph Adjacency List for Generic Data (weighted edge)

```
class Graph {
```

```
// Adj list
```

```
unordered_map<string,
list<pair<string, int>>> l;
```

```
public:
```

```
void addEdge(string x,
string y, bool bidir,
int wt) {
```

```
l[x].push_back(
make_pair(y, wt));
```

```
if (bidir) {
```

```
l[y].push_back(make_pair(x, wt)); }
```

```
void print() {
```

```
for (auto p : l) {
```

```
string city = p.first;
```

```
list<pair<string, int>> nbrs = p.second;
```

```
cout << city << " -> ";
```

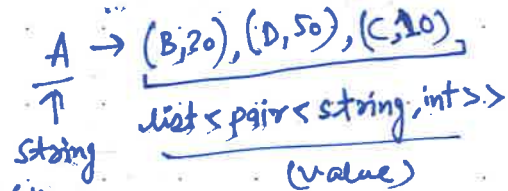
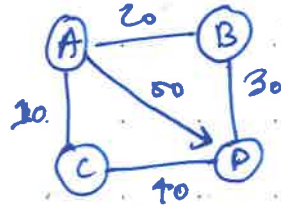
```
for (auto nbr : nbrs) {
```

```
string dest = nbr.first;
```

```
int dist = nbr.second;
```

```
cout << dest << " -> dist << " , ";
```

```
cout << endl; }
```



```
int main() {
```

```
Graph g;
```

```
g.addEdge("A", "B", true, 20);
```

```
g.addEdge("B", "D", true, 40);
```

```
g.addEdge("A", "C", true, 10);
```

```
g.addEdge("C", "D", true, 10);
```

```
g.addEdge("A", "D", false, 50);
```

directed edge

undirected = true
if false then directed edge

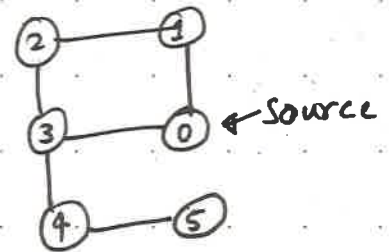
weight of edge.

Breadth First Search

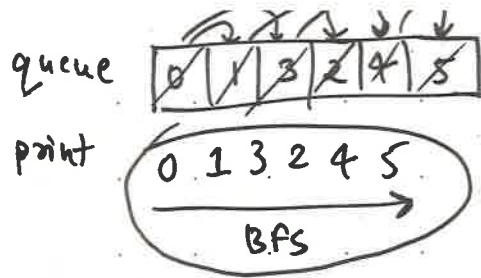
(Iterative)

Traversal through all the nodes of the graph.

Time Complexity:
 $O(V+E)$



- In this, we maintain a queue and start traversing from source till we print all nodes.
- (1) We push 0 in queue and then push its neighbours 1 & 3 and after pushing its neighbours we pop and print 0.
- (2) We also maintain a visited array/hash map which is true for visited nodes and false for unvisited ones. So that redundant nodes don't get printed. And so on till queue becomes empty.



0	0
1	1 3
2	2 4
3	5

Level wise print happens in BFS (acc. to neighbours)

Code (For undirected graph)

```
template <typename T>
```

```
class Graph{
```

```
map<T, list<T>> l;
```

```
public:
```

```
void addEdge(int x, int y){
```

```
l[x].push-back(y);
```

```
l[y].push-back(x);}
```

```
void bfs(T src){
```

```
map<T, int> visited; // stores the info about visited/unvisited nodes
```

```
queue<T> q;
```

```
q.push(src);
```

```
visited[src] = true;
```

```
while(!q.empty()){
```

```
T node = q.front();
```

```
q.pop();
```

```
for(int nbr : l[node]){
```

```
if(!visited[nbr]){
```

```
q.push(nbr);
```

```
visited[nbr] = true; // mark that neighbour as visited
```

```
}
```

```
int main(){
```

```
Graph<int> g;
```

```
(0,1);
```

```
(1,2);
```

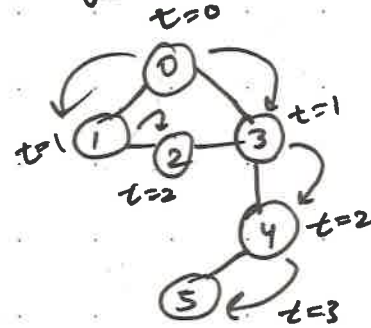
```
(2,3) -> (0,3)
```

```
(3,4);
```

```
(4,5);
```

```
}
```

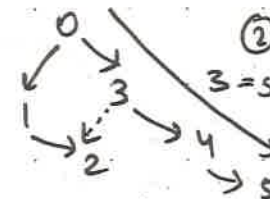
Single Source Shortest Path using BFS Graphs



To find the shortest distance from a source node to a specific node in an unweighted graph, BFS can be used.

(t → defines level of BFS Traversal)

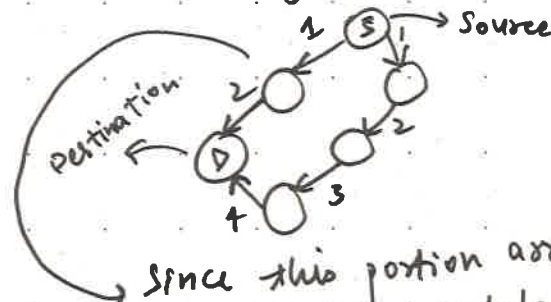
0	0
1	1, 3
2	2, 4
3	5



Either we can ~~print~~ push ② as neighbour of ① or ③. Both will give ② at level (t=2)

3 = shortest distance b/w 0 & 5

How do we get shortest distance using BFS. Example:



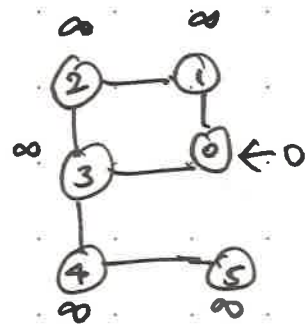
Since this portion arrives first so they will already update the visited map and t=2 for ④, and longest path won't be able to point ④. because it has already been visited.

Code

```

bfs ( ) {
    map<T, int> dist;
    queue<T> q;
    for (auto node-pair: l) {
        T node = node-pair.first;
        dist[node] = INT_MAX;
    }
    q.push(src);
    dist[src] = 0;
    while (!q.empty()) {
        T node = q.front();
        q.pop();
        for (int nbr: l[node]) {
            if (dist[nbr] == INT_MAX) {
                q.push(nbr);
                // mark this nbr as visited
                dist[nbr] = dist[node] + 1;
            }
        }
        for (auto node-pair: l) {
            T node = node-pair.first;
            int d = dist[node];
            cout << "Node " << node << " Dist from Src "
                << d << endl;
        }
    }
}

```



Set all the nodes to ∞ .

undirected

```

l[0] → 1, 3
l[1] → 2, 0
l[2] → 1, 3
l[3] → 0, 2, 4
l[4] → 2, 5
l[5] → 4

```

// Printing distance

```

for (auto node-pair: l) {
    T node = node-pair.first;
    int d = dist[node];
    cout << "Node " << node << " Dist from Src "
        << d << endl;
}

```

OP:- Node 0 Dist from Src 0
 Node 1 Dist from Src 1
 Node 2 Dist from Src 2
 Node 3 Dist from Src 2
 Node 4 Dist from Src 1
 Node 5 Dist from Src 3

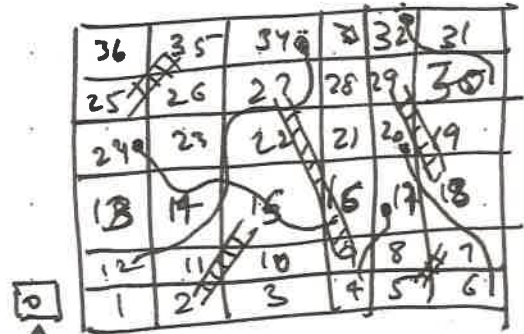
Snakes & Ladder BFS - SSSP Problem

Min. no. of Dice Throws Required to reach the destination 36 starting from the source. Find the shortest path as well.

~~we will make~~

if we reach snake posⁿ we goes back ~~to~~ by a value.

if we reach below a ladder we moved up by a value.



Snakes →

```

12 34
4 17
16 24
6 20
30 32

```

Ladders →

```

2 15
9 27
18 29
25 35

```

For ex:-

- If dice's value comes out to be 1 from posⁿ ① then ~~we~~ we go to ② & move up at ⑮. So ① → ⑮ will be an edge of graph.
 - if we were to be at ③④ then Snake will take us down to ⑮. So edge ③④ → ⑮ will be there.
- ∴ Directed graph will be made & with unweighted edges

We will just do little modifications in the bfs func.

```
bfs(T src, T dest) {
    // what value (bfs) it will print (min/shortest distance from src to dest)
    /* same func */
    cout << dist[dest];
}
```

int main() {

int board[50] = {0};

board[2] = 13;

board[5] = 2;

board[9] = 18;

board[18] = 11;

board[25] = 10;

board[17] = -13;

board[20] = -14;

board[24] = -8;

board[32] = -2;

board[34] = -22;

Graph <int> g;

for (int i = 0; i < 36; i++) {

for (int dice = 1; dice <= 6; dice++) {

int j = i + dice; // store posⁿ after dice throw

j += board[i];

if (j <= 36) {

g.addEdge(i, j);

}

g.addEdge(36, 36);

Now for (auto nbr: l[node]) {
} in bfs func
will also be able to
change value dist[36] ✓

// stores final posⁿ after climbing ladder or sliding through snake.

Node 36 was not added in l[] that's why not edge.

This makes up the whole snake game graph.

g.bfs(0, 36);
src dest

Min No. of dice thrown to reach 36 is 4.



To print shortest path :-

map <T, T> parent;

whenever we update node we do
parent[src] = src; → when we set distance of 0.

parent[nbr] = node; → for other nodes.

// dest to src.

T temp = dest;

while (parent[temp] != src) {

cout << temp << "←";

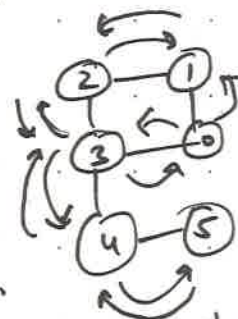
temp = parent[temp];

cout << src << endl;

Depth First Search (DFS)

In this we call recursion on neighbors nodes of source and keep printing till we have no further unvisited neighbour nodes.

For ex: 0 called 3 & 1
4 and 2 calls 0 and 1 and no unvisited neighbours left
calls 5 and no neighbours so return



if depends
0 → 1, 3
0 → 3, 1
which in first in order

```
template < typename T >
```

```
class Graph {
```

```
map < T, list < T > > l;
```

```
public:
```

```
void addEdge (int x, int y) {
    ...
}
```

```
void dfs-helper (T src, map < T, bool > &visited) {
```

```
    cout << src << " ";
```

```
    visited[src] = true; → After print mark it visited
```

```
    // go to all nbr of the node that's not visited
```

```
    for (T nbr : l[src]) {
```

```
        if (!visited[nbr]) {
```

```
            dfs-helper(nbr, visited); } }
```

```
void dfs (T src) {
```

```
    map < T, bool > visited;
```

```
    // Mark all the nodes as not visited in the beginning
```

```
    for (auto p : l) {
```

```
        T node = p.first;
```

```
        visited[node] = false; }
```

```
    // call helper func
```

```
    dfs-helper(src, visited);
```

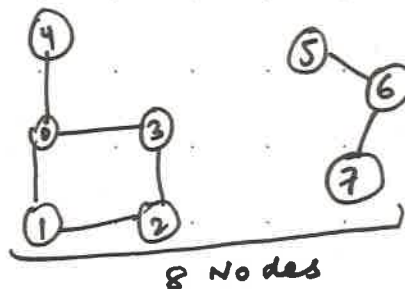
```
};
```

```
int main() {
```

```
    ...
```

```
    ...
```

Connected Components using DFS Graphs (undirected)



8 Nodes

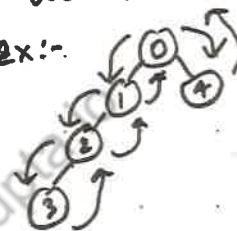
There are two connected components in above undirected graph

1) 0 1 2 3 4

2) 5 6 7

When we do dfs traversal of each components we traverse like its a tree.

ex:-



∴ They are called DFS Tree and all the ~~same~~ connected components is DFS Forest.

We will have to call dfs for each connected Component. We will modify dfs func.

⇒ void dfs () {

```
    map < T, bool > visited;
```

```
    for (auto p : l) {
```

```
        T node = p.first;
```

```
        visited[node] = false; }
```

```
    int cnt = 0;
```

```
    for (auto p : l) {
```

```
        T node = p.first;
```

```
        int cnt = 0;
```

```
        if (!visited[node]) {
```

```
            cout << "Component " << cnt << " -->";
```

```
            dfs-helper(node, visited);
```

```
            cnt++; → Counts no. of connected components
```

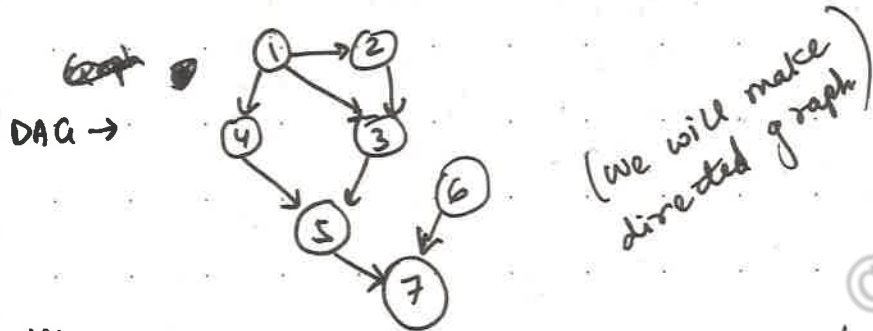
we run dfs-helper for each node considering it as src node. if node is visited no need to call

"There's nothing more important than our good health - that's our principal capital asset."

o/p:- Component 0 $\rightarrow$ 0 1 2 3 4
 Component 1 $\rightarrow$ 5 6 7
 (We can also use BFS in this)

Directed Acyclic Graph's Topological Sort using DFS

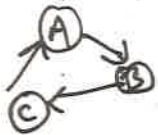
Topological sorting for DAG is a linear ordering of vertices such that every directed edge u, v ; vertex u comes before v in the ordering. Topological sorting for a graph is not possible if the graph is not DAG (i.e. cyclic graphs)



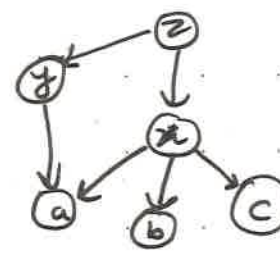
(we will make directed graph)



Doesn't work graphs (cyclic)



Observation
 (DP also works like this) - Find values from general prev. value in linear ordering



x, y, x, c, b, a
order

• Do modification in bfs and bfs-helper.
 (we again have to do DFS ~~for~~ taking each node as src so that no single nodes left as dependencies)
 void dfs-helper(src, &visited, &ordering)

```
visited[src] = true;
for (T nbr: l[src]) {
  if (!visited[nbr]) {
    dfs-helper(nbr, visited, ordering);
  }
  ordering.push-front(src);
  return;
}
```

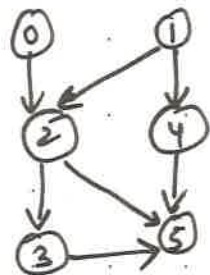
we will not print just push it in queue and print all elements in last. [Refer gfg for algo]

(push from front)

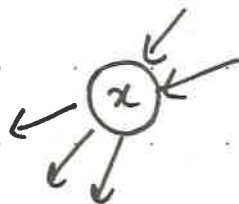
void dfs() {
 list<T> ordering;
 for (auto p: l) {
 T node = p.first;
 if (!visited[node]) {
 dfs-helper(node, visited, ordering);
 }
 }
 for (auto node: ordering) {
 cout << node << endl;
 }
}

Not complete code (modification)

Topological sort using BFS



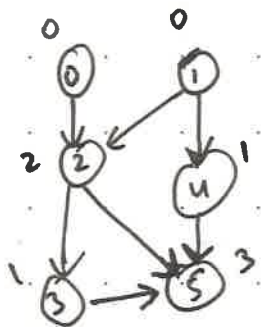
We need to find a node which has no dependency.



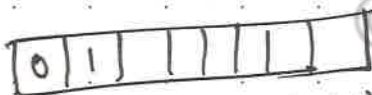
Indegree: No. of incoming edges to x .
Out degree: No. of outgoing edges from x .

Algo.

We just have to find in degree of each node.

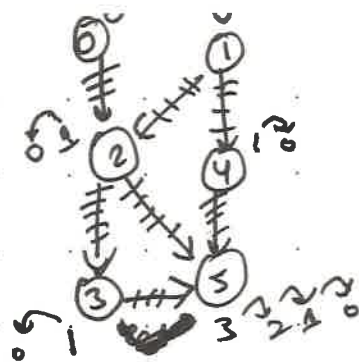


So we will take queue



and push nodes with 0 indegree. and one by one

we start popping element ~~and pushing their neighbours~~. and while popping we also will decrease the indegree of neighbours. If while decreasing the indegree of any node becomes 0 then we push it in queue.



Print $\rightarrow$ 0 1 2 4 3 5

Space $\uparrow$
(Use of an extra queue)

Code ~~(C++)~~

~~class graph {~~

~~int n;~~
void topological_bfs()

map<T, int> indegree;

for(auto node : l) {

T nbr = node.first;

indegree[nbr] = 0;

// update indegree by traversing edges $x \rightarrow y$

// indegree[y]++

for(auto p : l) {

int x = p.first;

for(auto y : p.second) {

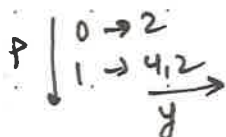
indegree[y]++;

}

// bfs

1. // step find 0 indegree nodes

queue<int> q;



for each value of neighbours we ~~is~~ increment their indegree.

~~for(int i=0; i<v; i++)~~

```
for(auto p: l){
    if (indegree[p.first] == 0){
        q.push(p.first); } }
```

// start removing elements from the queue

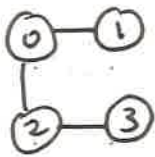
```
while(!q.empty()){
```

```
    int node = q.front();
    q.pop();
```

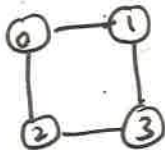
// iterate over nbrs of that node & reduce their indegree.

```
for(auto nbr: l[node]){
    indegree[nbr]--;
    if (indegree[nbr] == 0){
        q.push(nbr); }
```

Undirected Graph is a Tree or Not (Check)



Acylic $\therefore$ Tree

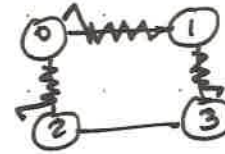


Cyclic $\therefore$ Not a Tree

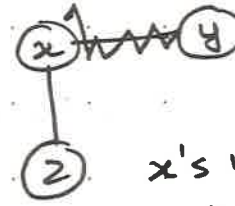
• check if we are visiting a node by more than one path. if there exist ~~per~~ multiple paths ~~via~~ which we can reach there. Then the undirected

graph is cyclic.

For ex:-



Corner case



x's nbr
= y, z

(but y is parent of z so won't be considered in counting of no. of paths to prove cycle) and z is not visited so acyclic.

① nbrs $\rightarrow$ 0 & 3

so in this case it moves towards ① and mark it visited. and then mark visited for other nbr

③. Then for ①, it marks ② visited but when ③ checks for its nbr which is ② its already visited. (and ② is not parent of ③) so in this case there exist cycle.

Code

~~make~~ DIR overhead codes for undirected graph

bool isTree() $\rightarrow$ T src

map<T, bool> visited;

~~map~~ map<T, T> parent;

queue<T> q;

$\rightarrow$ We will use BFS

```
for(auto p: l){
```

```
    T node = p.first;
```

```
    visited[node] = false;
```

```
    parent[node] = node; }
```

```

q.push(src);
visited[src] = true;
// pop out one node and visit its nbrs.
while(!q.empty()){

```

```

    T node = q.front();
    q.pop();

```

```

    for(auto nbr : l[node]){
        if(visited[nbr] == true and parent[node] != nbr){

```

```

            return false;

```

```

        } else if (!visited[nbr]){

```

```

            visited[nbr] = true;
            parent[nbr] = node;
            q.push(nbr);

```

(if all nbrs of node is ~~not~~ not visited then we continue ahead)

if nbr of the node other than its parent is already visited then cycle exists.

```

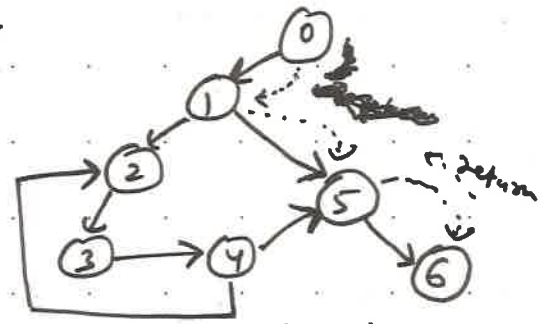
        }
    }
    return true;
}

```

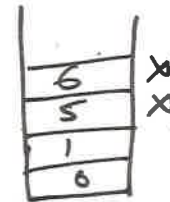
if no cycle then its tree (acyclic)

Cyclic Detection in Directed Graph using DFS

We will maintain our current path through a stack and a visited mapping.



Back Edge (edge to an ancestor of that node)



stack (for current path)

From 0 we go to 1 and the node we visit we push it to stack. Then from 1 to 5. 5 to 6.

Now no further we can go so we start returning and popping the same element from stack too.

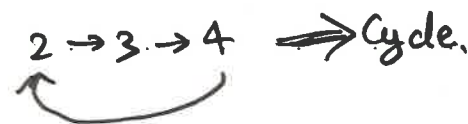
When we return till 1 we can move to 2 and 2 to 3, 3 to 4.

From 4 we can't go to 5 as ~~into stack~~ 5 is visited already and also not in current path (stack) [No cycle here] but from 4 we can go back to 2 again which is in current path (stack).



stack (current path)

* So cycle is detected here.



Q How would be available to find whether that node exist in stack (current path) in $O(1)$?

1. Using array (implemented on the basis of stack)

i.e.

	T			
0	1	2	3	4

 1 is present.
($\therefore \text{stack}[1] == \text{true}$)

Code

DIY overhead for directed graph (You can also use (directed + undirected) overhead func. for generic & func.

```
bool cycle-helper (int node, &visited, &stack) {
```

```
    visited[node] = true;
```

```
    stack[node] = true;
```

```
    for (auto nbr : l[node]) {
```

```
        if (stack[nbr] == true) {
```

```
            return true; }
```

```
        else if (visited[nbr] != false) {
```

```
            bool cycle-mila = cycle-helper (nbr, visited, stack);
```

```
            if (cycle-mila == true) {
```

```
                return true; } }
```

```
// either we reached dead end or no cycle in current path
```

```
stack[node] = false;
```

```
return false; }
```

→ if this nbr exist in current path already then return true

use check or further whether there exist cycle

```
bool contains_cycle () {
```

```
    map < T, bool > visited;
```

```
    map < T, bool > stack;
```

```
    return cycle-helper (node, visited, stack);
```

```
}
```

```
for (auto p: l) {
```

```
    T node = p.first;
```

```
    visited[node] = stack[node] = false;
```

```
}
```

Cycle Detection Undirected Graph using DFS

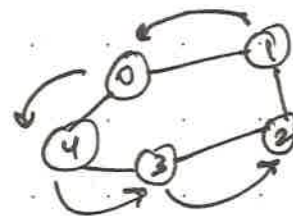
if we start from ①

① → ④ → ③ → ②

②

At ② we see every instant we check if neighbour of this node is visited and not the parent of this neighbour.

For ex :- For ②, neighbours are ① & ③ but ③ is parent of ② but ① is not the parent of ② & also visited so cycle exist.



cycle
if more than 1 way to visit any node

Code

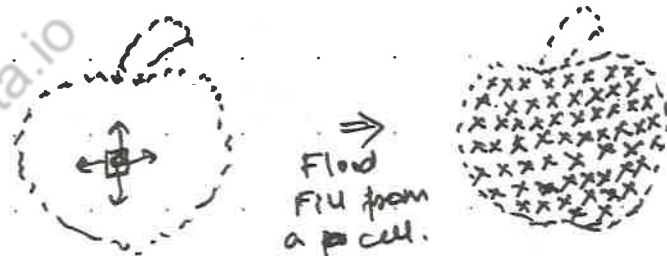
```
bool cycle-helper(int node, bool &visited, int parent){
    visited[node] = true;
    for(auto nbr: l[node]){
        if(!visited[nbr]){
            bool cycle_mila = cycle-helper(nbr, visited,
                                           node);
            if(cycle_mila){
                return true;
            }
        }
        else if (nbr != parent){
            return true;
        }
    }
    return false;
}
```

only a variable enough to store parent

```
bool contains-cycle(){
    bool map < T, bool > visited;
    for(auto p: l){
        T node = p.first;
        visited[node] = false;
    }
    return cycle-helper(0, visited, -1);
}
```

Flood Fill Algorithm

- Flood Fill is a simple variant of BFS/DFS that can be used to label (colour) the various connected components present in a graph.
- It is generally performed on implicit graphs (2D matrices)
- Starting from a particular cell we call 'DFS' on the neighbouring cells to colour them. Neighbours can be '4' (↑, ↓, ←, →) or '8' if we include diagonals.



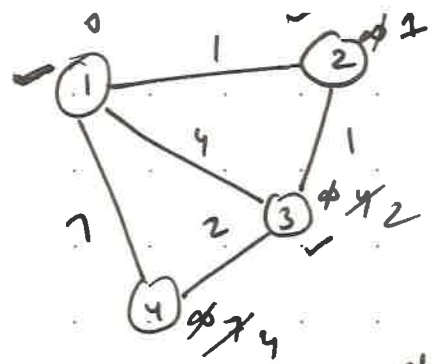
Code

```
int R;
int C;
void printmat(char input[][50]){
    /* prints the matrix */
}

int dx[] = {-1, 0, 1, 0};
int dy[] = {0, -1, 0, 1};
f(i+dx, j+dy)
} defines directions for each cell

void FloodFill(char mat[][50],
               int i, int j, char ch, char color){
    // Base case
    if (i < 0 || j < 0 || i >= R || j >= C){
        return;
    }
}
```

Set $\rightarrow$ min element can be found in $O(1)$
(keeps sorted) $\downarrow$ (distance in this Q)
& unique and removal - $O(\log n)$



set (remain sorted)

Dist	1	4	7
Node	2	3	4

after updating
we pop this because
it got the min distance
from src.
also remove & insert
updated.

2	7
3	4

↓
distance minimised
and finalized. Remove it.

Distance of ④ via ③ is
 $2 + 2 = 4$

4
4

 → Finalized. Remove
Now set is empty

Code

```
class Graph {
    unordered_map<T, list<pair<T, int>>> m;
public:
    void addEdge(T u, T v, int dist, bool bidir = true) {
        m[u].push_back(make_pair(v, dist));
        if (bidir) {
            m[v].push_back(make_pair(u, dist));
        }
    }
}
```

void printAdj()

```
{
    for (auto j : m) {
        cout << j.first << "→";
        for (auto l : j.second) {
            cout << "(" << l.first << ", " << l.second << ") ";
        }
    }
}
```

void dijkstraSP(T src)

```
{
    unordered_map<T, int> dist;
```

1	2	3	4
0	∞	∞	∞

```
for (auto j : m) {
    dist[j.first] = INT_MAX;
}
```

set<pair<int, T>> s;
we took distance first because set sorts wrt 1st parameter.

dist[src] = 0;

s.insert(make_pair(0, src));

while (!s.empty()) {

auto p = *(s.begin()); → picking up first element from set
T node = p.second;

int nodeDist = p.first;

s.erase(s.begin());

// Iterate over neighbours of current node

for (auto childPair : m[node]) {

Dijkstra condⁿ ← if (nodeDist + childPair.second < dist[childPair.first]) {

// In the set updation of a particular is not possible

// We have to remove the old pair and insert the new pair to simulation updation

```

1 dest = child.first;
and f = s.find(make_pair(dist[dest], dest));
if (f != s.end()) {
    s.erase(f);
}

```

// Insert new pair

```

dist[dest] = nodeDist + childPair.second;
s.insert(make_pair(dist[dest], dest));
}
}

```

```

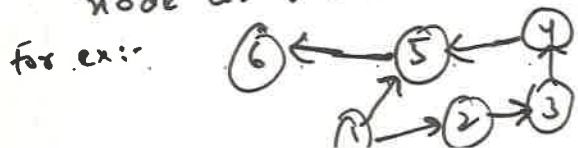
for (auto d : dist) {
    cout << d.first << " is located at distance of "
    << d.second << endl;
}
};

```

O/P:- 2 is located at dis of 0
 1 4
 4 2
 3

of all nodes
 any question where you have to find distance in directed graph

NOTE:- In case of a directed graph if there is a calculation SSSP (BFS) or Dijkstra (SSSP) then maybe the destination node is not included in adjacency list.



A NEW-AGE COMPREHENSIVE HEALTH INSURANCE



Covers up to 6 family members



30-Day Pre & 60-Day Post Hospitalization



No-Claim Bonus up to 150%



540+ Day Care Treatments



Annual Health Check-up for all Insured members

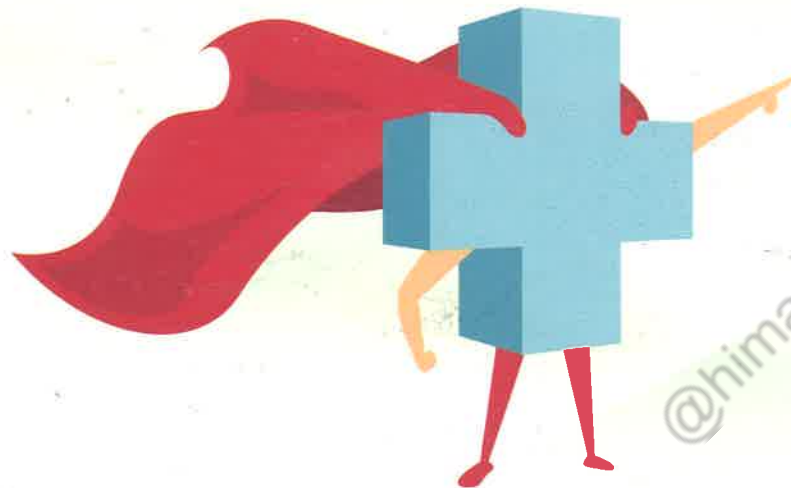


Unlimited Automatic Recharge of Sum Insured

Conditions Apply

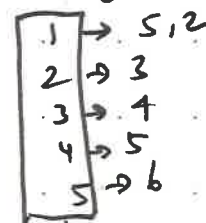
Care

THE SUPERHERO OF HEALTH INSURANCE



Conditions Apply

Adjacency list



~~map~~
l.first

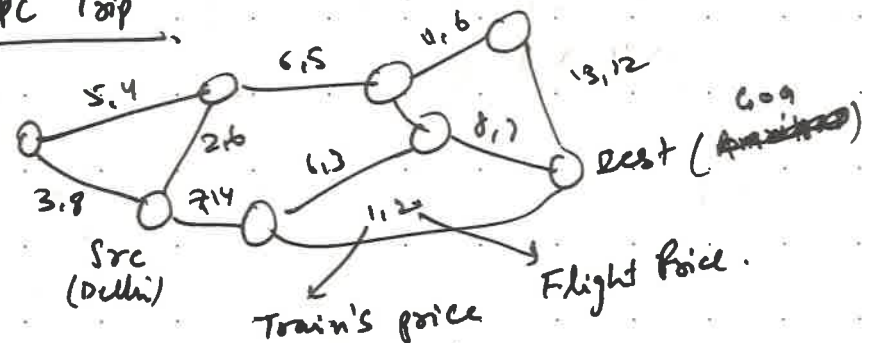
Now when you update everyone's distance in ~~map~~ map (dist) then
for (auto &r : l) {

T node = r.first;
cout << dist[node]; }

but 6 is not here. so this loop will not print for 6 so we need to make a self edge
g.addEdge(6,6);

Time complexity → $(E \log V)$ → Dijkstra's

ICPC Trip



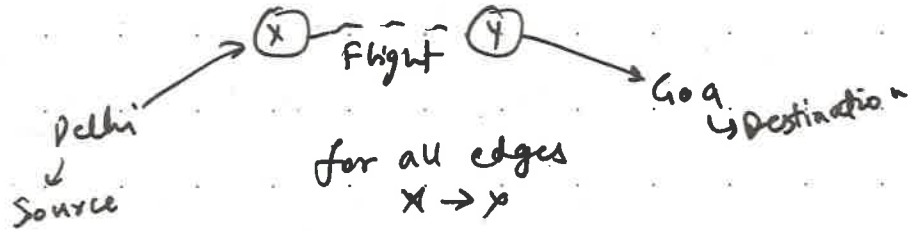
Minimize the total fare
① Almost 1 flight in your whole trip (optional)
0 Flight / 1 Flight

Algo

(0 flight)
1) If you take no flight but only trains.
src —————> Goa (Apply dijkstra's directly)

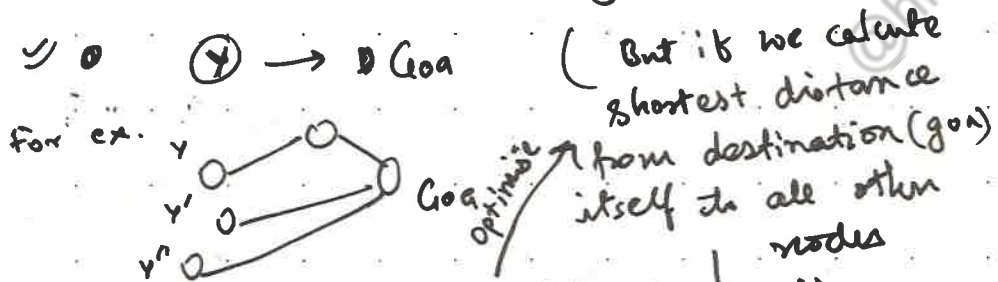
We will have ~~an~~ an array where we will have a shortest distance of all nodes from ~~src~~ $\rightarrow$ $dist1[]$

2) If we take 1 flight in trip



$\Rightarrow$ Delhi $\rightarrow$ (X) ($\forall x \in V$) (Dijkstra's will give shortest distance of all other nodes from src)

$\Rightarrow$ $x \rightarrow y \Rightarrow$ flight price array



if we calculate shortest distance from each y node to Goa then $O(V \log V)$ for all vertex y, y', y'' etc,

then only one time we will apply Dijkstra's and get $dist2[]$ for

No. of times Dijkstra's applied is 2 times

① Delhi as src $\rightarrow$ $dist1$

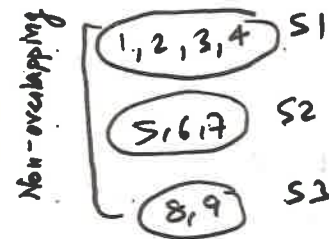
② Goa as src $\rightarrow$ $dist2$

$$ans = \min_{\forall x, y} \left[\begin{aligned} &\min \left[dist1[x] + f_c(x, y) + dist2[y] \right] \\ &\text{flight fare from } x \text{ to } y \end{aligned} \right]$$

$\rightarrow$ $dist[src \rightarrow dest]$

- Disjoint Union Set / (Union-Find)

Usage $\rightarrow$ cycle finding in undirected graph, Kruskal's Algo



Find (8) $\rightarrow$ S3

Find (6) $\rightarrow$ S2

Union (S2, S3) $\rightarrow$ {5, 6, 7, 8, 9}

A disjoint-set data structure that keeps track of a set of elements partitioned into a no. of disjoint (non-overlapping) subsets. A union-find algo is an algo that performs two useful operations on such data str.

find :- Determine which subsets an element is in. This can be used to find two elements is in same subsets or not.

Union :- Join two subsets into single subset.

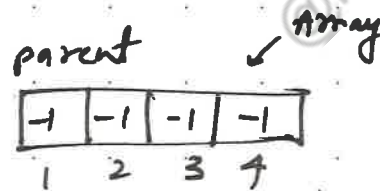
- Union-Find Algo can be used to ~~take~~ check whether an undirected graph contains cycle or not.

$O(N) \rightarrow$ Union-Find

↓ optimised

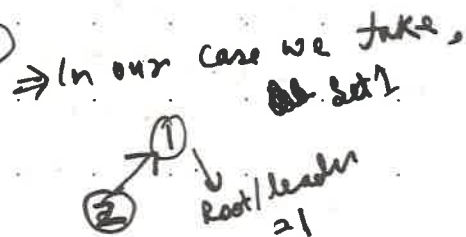
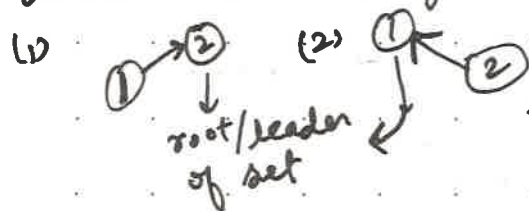
$O(1) \rightarrow$ Path compression
→ Union by Rank

Union & Find operations

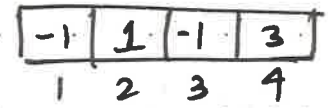
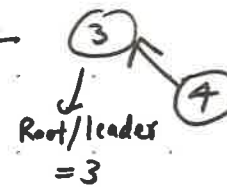


Initially, no parent for any node.

Union (1,2) → We can do union in two ways:-



⇒ Union (3,4) - Set 2



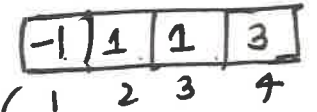
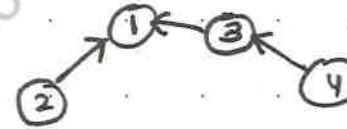
It will store ~~parent~~ parent of its set.
i.e. parent of 2 is 1 and that of 4 is 3.

$s1 = \text{find}(1) = 1$

$s2 = \text{find}(4) = 3$

Find(x) gives off the root/leader of set in which x is in.
if $(s1 \neq s2) \{ \text{parent}[s2] = s1 \}$

⇒ Union (1,4)



• 1 is root/leader of new set being made.

• 1 is also parent of 2 & 3.

• 3 is parent of 4.

$s1 = \text{find}(2) = 1$

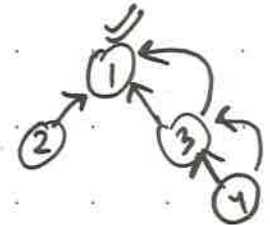
$s2 = \text{find}(3) = 1$

Since $(s1 == s2)$ then making ~~edge~~ ^{edge} between 2 & 3 is possible and thus

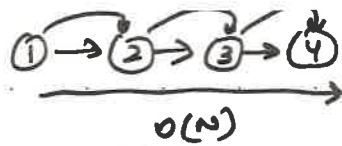
Cycle is present.

Find(x) pseudocode

```
int find(int i) {
    if (parent[i] == -1) {
        return i;
    }
    return find(parent[i]);
}
```



Worst case :- $O(N)$



Union Pseudocode

```
void union (int a, int b) {
```

```
    s1 = find(a);
    s2 = find(b);
    if (s1 != s2) {
        parent[s2] = s1;
    }
}
```

Return leader of a, b

non overlapping sets

parent array

Worst case :- $O(N)$

Cycle Detection in an undirected Graph using DSU

```
class Graph {
```

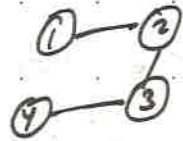
```
    int V;
    list<pair<int, int>> l;
```

public:

```
    Graph(int V) {
        this->V = V;
    }
```

```
    int addEdge(int u, int v) {
        l.push_back(make_pair(u, v));
    }
```

```
    bool contains-cycle() {
        int findSet(int i, int parent[]) {
            if (parent[i] == -1) {
                return i;
            }
        }
    }
```



(1,2) (2,3) (3,4)

Edge list
(we will use)
to store edges

```
return findSet (parent[i],
parent);
}
```

// Union

```
void union-set (int x, int y,
int parent[]) {
```

```
    int s1 = findSet(x, parent);
    int s2 = findSet(y, parent);
```

```
    if (s1 != s2) {
        parent[s1] = s2;
    }
```

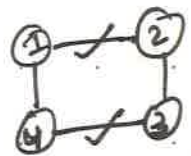
```
bool contains-cycle() {
```

```
    int *parent = new int[V];
    for (int i=0; i<V; i++) {
        parent[i] = -1;
    }
```

// Iterate over the edge list

```
for (auto edge : l) {
    int i = edge.first;
    int j = edge.second;
    int s1 = findSet(i, parent);
    int s2 = findSet(j, parent);
```

```
    if (s1 != s2) {
        union-set(s1, s2, parent);
    }
```



Union(1,2)

Union(3,4)



Union(2,3)



When we do
union of 1 & 4
s1 == s2 comes off
∴ cycle exists
s1 = find(1) = 1
s2 = find(4) = 1

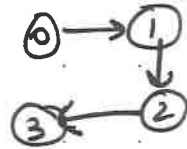
if you try
to unite two
nodes already
in same path (
have same leader)
then you
make cycle.

"Health is the thing that makes you feel that now is the best time of the year."


```

else {
    return true; } // You found out a cycle
    while checking
    if union
    is possible or not
    delete [] parent;
    return false; }
};

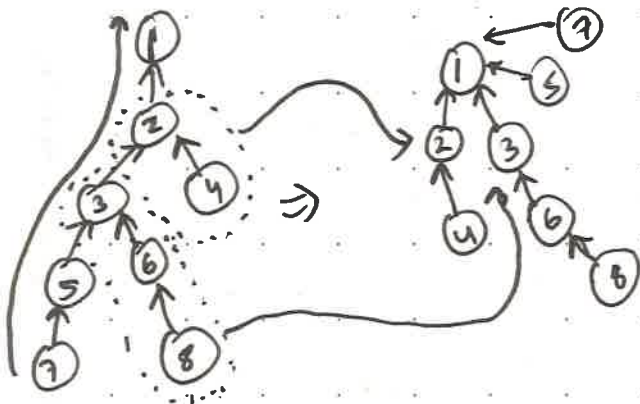
```



when union (0,3) it returns true because 0 & 3 both have leader

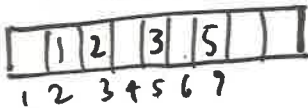
Path Compression

[optimisation for Find Func.]

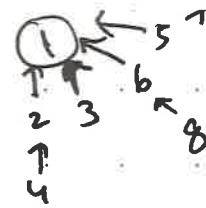


We ~~will not~~ ~~do anything~~ will not do anything with current's nodes childrens.

find(7) = 1 → when we called find(7) path compression compresses the all the nodes from 7 till leader of this current path



find(6) ⇒
we compress the path from 6 to leader



Pseudo code

```

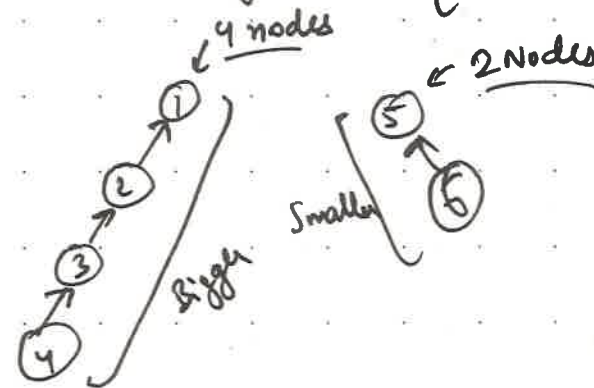
int find(int i) {
    if (i == -1) return i;
    return parent[i] = find(parent[i]);
}

```

↑
current node will now point directly to leader.
it will return leader of current path.

Union By Rank

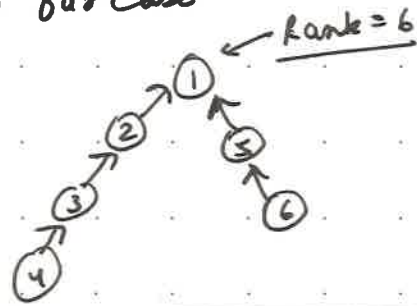
[optimisation for Union Func.]



An array will store the rank of each set.
So in this, instead of just making parent[5] = 1.

We will first check which set's rank is greater and then decide ~~whose~~ whose leader will be the parent.

In our case



Rank-value[] =

6					
1	2	3	4	5	6

After both the optimisation the time complexity almost becomes $O(1)$ for DSU opⁿ.

```
void union-Set (int x, int y, int parent[],
               int rank[]) {
```

```
    int s1 = findSet (x, parent);
```

```
    int s2 = findSet (y, parent);
```

```
    if (s1 != s2) {
```

```
        if (rank[s1] < rank[s2]) {
```

```
            parent[s1] = s2;
```

```
            rank[s2] += rank[s1]; }
```

```
        else {
```

```
            parent[s2] = s1;
```

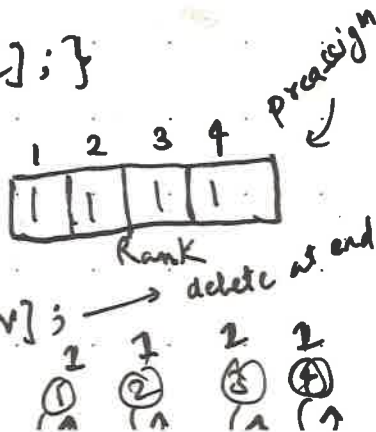
```
            rank[s1] += rank[s2]; }
```

```
    }
```

bool Contains cycle () {

```
    int *rank = new int[V];
```

```
    for (int i = 0; i < V; i++) {
```



```
int main() {
```

```
    Graph g(7);
```

```
    g.addEdge(0, 1);
```

```
    g.addEdge(1, 2);
```

```
    g.addEdge(2, 3);
```

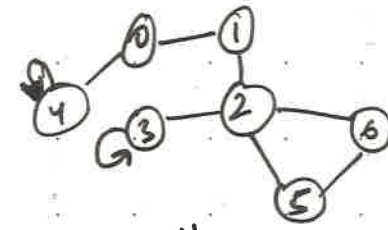
```
    (0, 4);
```

```
    (5, 6);
```

```
    (2, 5);
```

```
    (2, 6);
```

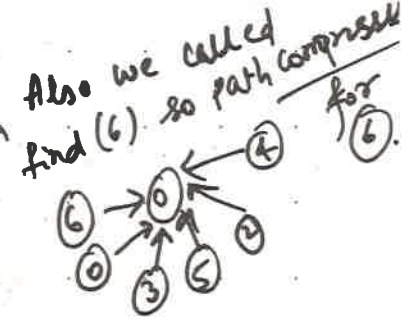
```
}
```



Adding this edge (5 & 6) both don't lie in current DSU graph.

leader of 2 = 0
leader of 6 = 0

So making edge (2, 6) will result in cycle.



Parent array =

-1	0	0	0	0	0	0
0	1	2	3	4	5	6

Rank array =

7	1	1	1	1	1	2
0	1	2	3	4	5	6

2 because we don't change but change for only leader nodes.

Pairing Problem

N cities numbered 0 to N-1. Choose 2 cities such that there is no path from first city to 2nd city. You r. given info in the form of M pairs (x,y). i.e there will be undirected road b/w city x & city y. Find out the no. of ways in which cities can be chosen.

For ex:-
 ① 3 Nodes
 ② 2 Nodes
 0 1 P:-
 6

Input:-

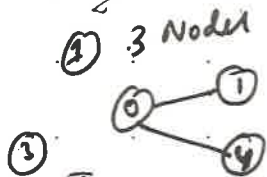
5 3

0 1

2 3

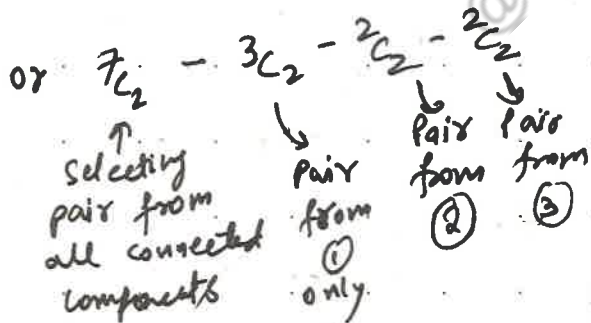
0 4

5 6



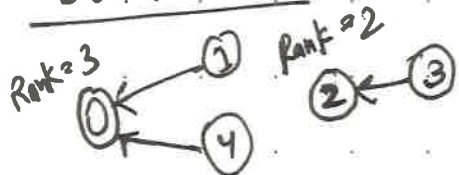
2 Nodes
 Connected Components graph

We know that we can't pair from Component because we want pair with no path b/w them.



x y
 3 x 2 [1 & 2]
 3 x 2 [1 & 0]
 2 x 2 [2 & 3]
 ⇒ 6 + 6 + 4 = 16

DSU Method



if we choose a node and check how many pairs it can have

$$① \rightarrow 2, 3 \Rightarrow \text{rank}(\text{find}(0)) = 0 \Rightarrow 5 - 0 = 5$$

$$① \rightarrow 2, 3 \Rightarrow \text{rank}(\text{find}(1)) = 1 \Rightarrow 5 - 1 = 4$$

Total nodes in this component

$$④ \Rightarrow \text{rank}(\text{find}(4)) = 3 \Rightarrow 5 - 3 = 2$$

$$② \Rightarrow \text{rank}(\text{find}(2)) = 2 \Rightarrow 5 - 2 = 3$$

$$③ \Rightarrow 5 - 2 = 3$$

Total such pairs = 2 + 2 + 2 + 3 + 3 = 12
 since pair repeats (0, 2) & (2, 0)

$$\therefore \frac{12}{2} = 6 \text{ pairs.}$$

hard

easy pairing!

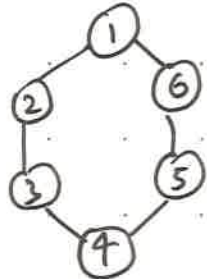
Minimum Spanning Tree

Given a connected and undirected graph, a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together. many spanning trees are possible for a graph.

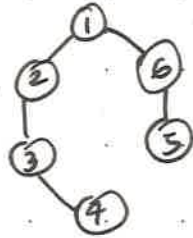
A MST or (min. weight spanning tree) for a weighted, connected & undirected graph is a spanning tree with least weight possible.

Application:- Network design.

for ex:- your office wants to set phone lines over different branch they have. So you want set a line in a min. cost possible so that it doesn't go out of budget.



$|E|=6, |V|=6$



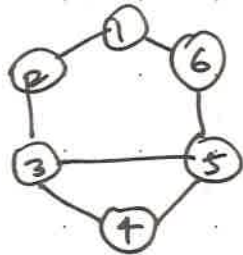
One of the spanning tree

(6 MSTs possible)

No. of edges in S.P.s and MSTs

$$|E| = |V| - 1 = 5$$

$$|V| = 6$$



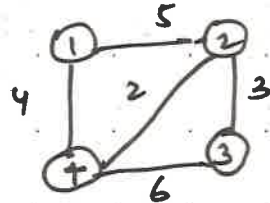
Graph with two cycles

Graph formula of no. of MST possible
 $= |E| - |V| + 1 - \text{no. of cycles}$

If we need to find min "cost" spanning tree

Two algos are there:

- (1) Prim's
- (2) Kruskal's



Prim's & Kruskal's $\Rightarrow$ Greedy Algo

At each step we go for (local minima) [local optimum] so that we get global optimum/minima.

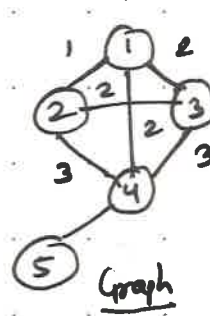
[Prim's & Kruskal's Algo is explained in DAA notes]

1) Kruskal's Algo

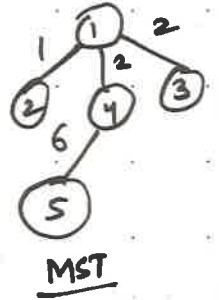
① Sort all edges based upon weight

Edge $x-y$	weight w
1-2	1
1-3	2
1-4	2
2-3	2
3-4	3
2-4	3
4-5	6

} form cycle



Graph



MST

② Pick the smallest edge, Add it to your MST if it doesn't form a cycle.

"The only way to keep your health is to eat what you don't want, drink what you don't like and do what you'd rather not."

Code

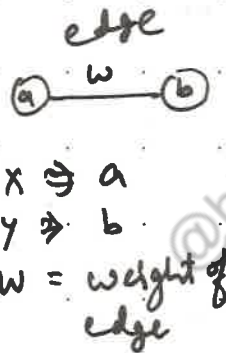
```
class DSU {
    int *parent;
    int *rank;
public:
    DSU (int n) {
        parent = new int [n];
        rank = new int [n];
        for (0 → n) {
            parent[i] = -1;
            rank[i] = 1; }
    find() {
    union() {
};
```

```
class Graph {
    vector <vector <int >> > edgelist;
    int V;
public:
```

```
    Graph (int V) {
        this → V = V; }
```

```
    void addEdge (int x, int y, int w) {
        edgelist.push_back {w, x, y}; }
    }
```

```
    int kruskal () {
        sort (edgelist.begin(), edgelist.end());
        DSU s(V);
```



```
int ans = 0;
for (auto edge : edgelist) {
    int w = edge[0];
    int x = edge[1];
    int y = edge[2];
    // take that edge in MST if it doesn't
    // form a cycle
    if (s.find(x) != s.find(y)) {
        s.unite(x, y);
        ans ans += w; }
    }
return ans; }
```

cost of
MST,

```
int main () {
    Graph g(4);
    cout << g.kruskal() << endl;
}
```

Prim's Algo

MST set = [] (we will maintain)
Active edge = edge from a vertex in MST set to V.
MST edge = an edge that has been included in MST so far.

Algo

1. start from any source vertex

2. Out of all active edges, pick the one with smallest weight.

3. ~~Edges starting from~~ ~~in the~~ ~~active edge list~~

→ We will maintain active edge list and vertex available set also.

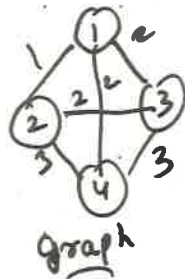
→ The smallest edge we picked



Insert the outgoing node y in MST set.

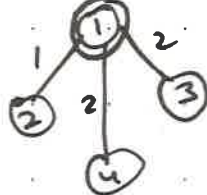
→ Now add ~~edges~~ neighbour edges of y in the active edge list.

3. Repeat from step 2 again.



⇒

We choose 1 as src, so active edges with 1 → Add it in MST



We select min weight edge 2.

and add 2 in MST set []

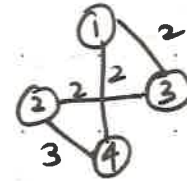
$V = [1, 2, 3, 4]$
MST = [1, 2]

MST ⇒ 4
so far



Active edges after adding. (~~add~~ add neighbour edges of 2 in active edge list)

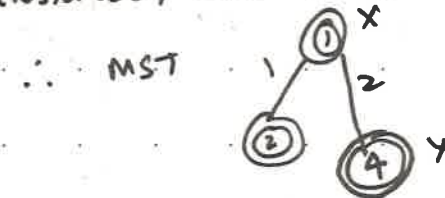
$V = [3, 4]$
MST = [1, 2]



Active edges so far

Select min out of all Active edges we have:

~~1-4~~, 1-3, 2-3 are active edges which are min. For instance, we choose 1-4



$V = [3]$

MST = [1, 2, 4]



Active edges

1 → 3
2 → 3
4 → 3

Add 4 in MST.

At every point we will also check if the two nodes ~~are~~ of ~~active~~ edge we are selecting should not be in (MST) only. [We want no cycle]
So we add only those edges which forms no cycle. [Ex:- 2 → 4 can't be taken]

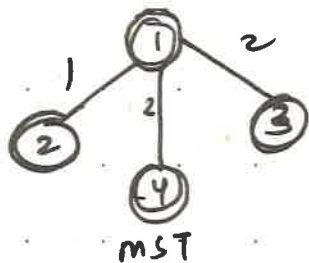
We will select ~~edge~~ active edge



and add $y = 3$ in MST set

V = []

MST = [1, 2, 4, 3]



MST = 1 + 2 + 2 = 5
Cost

Iterative approach $\rightarrow O(N)$

MinHeap $\Rightarrow O(\log N) \Rightarrow$ (Optimised)

Code. (Priority Queue (min heap) implementation)

class Graph {

~~map<int, pair<int, int>>~~

vector<pair<int, int>> *l;

int V;

public:

Graph(int n){

V = n;

l = new vector<pair<int, int>>[n]; }

void addEdge(int x, int y, int w){

l[x].push_back({y, w});

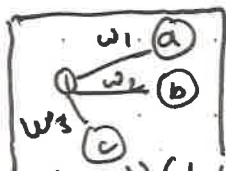
l[y].push_back({x, w}); }

int prim-mst(){

stores
active
edges

priority-queue<pair<int, int>, vector<
pair<int, int>>, greater<pair<
int, int>>>> Q;

bool *visited = new bool[V]{0}; // visited
nodes



1 $\rightarrow$ (a, w1)
2 $\rightarrow$ (b, w2)
3 $\rightarrow$ (c, w3)

int ans = 0;

Q.push({0, 0}); // {weight, node}

while (!Q.empty()) {

// pick out the edge with
min wt.

auto best = Q.top();

Q.pop();

int to = best.second;

int weight = best.first;

if (visited[to]) {

// discard the edge, and continue (cycle was
forming)

continue; }

// otherwise take the current edge

ans += weight;

visited[to] = 1;

// add the new edges (neighbours of this
node) as active edges in queue.

for (auto x : l[to]) {

if (visited[x.first] == 0) {

Q.push({x.second, x.first}); }

}

return ans;

}

because
priority queue
compares
acc. to first
argument.

we are selecting
active edge
with min wt.

Single Source Shortest Path Algos

BFS

→ Unweighted graphs
→ $O(V+E)$

Dijkstra

→ weighted
→ Positive weights
→ $O(E \log V)$

(It may or may not fail in case of negative weights)

Bellman

→ weighted graph
→ Directed
→ -ve weights

$O(VE)$
(takes more time than others but stable)

Interview

BFS & Dijkstra asked only

[NOTE :- Graph with N nodes & $N-1$ edges is tree]

Hamiltonian Cycle (Backtracking)

Hamiltonian path of an undirected graph is a path that visits each vertex exactly once.

A Hamiltonian cycle is a Hamiltonian path such that there is an edge from the last vertex of the to the first vertex of H.P. (we comes to starting node)

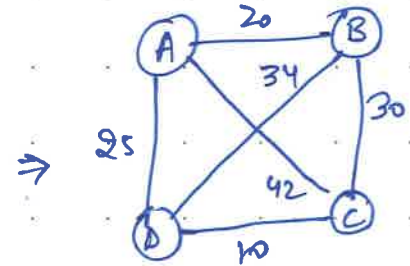
— DP with bitmask S:

Travelling Salesman Problem (2-d DP)

You are given a graph. You need to find min weight hamiltonian cycle. (The hamiltonian cycle which contains minimum weight sum).

I/P:-

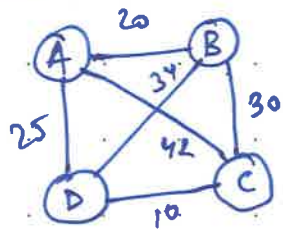
	A	B	C	D
A	0	20	42	25
B	20	0	30	34
C	42	30	0	10
D	25	34	10	0



Paths possible

A-B-C-D-A
B-C-D-A-B

Using dp (Top-down)



⇒ Store 'set of cities' visited so far in the current branch.

⇒ We will also maintain bit mask for no. of nodes visited.

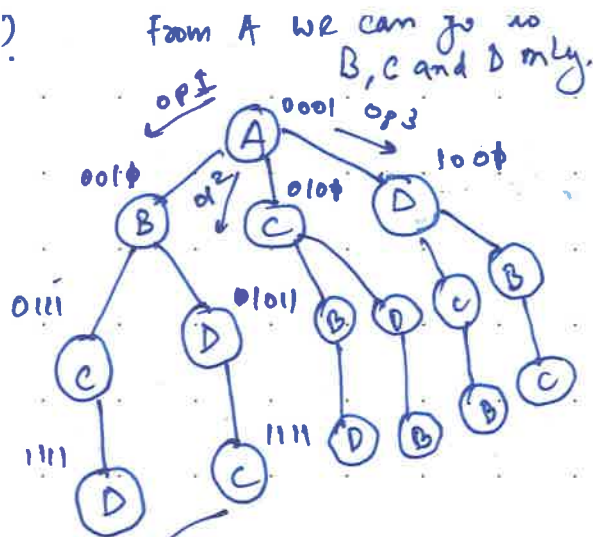
mask = 0 0 0 0

if all nodes are visited then

mask = 1 1 1 1

∴ Base case

if (visited_all) {
return cost[city][0];}



• So at each node we made branches according to ways it can go to which node first.

Base case when mask = 1 1 1 1. So when all nodes have been visited we go back to starting node which is A.

when we reach at D in base case we return from D → A and return 25 which is graph[city][0] so that it gets added to current-weight-sum of the path.

∴ For ex:-

$$\text{wt}(A-B-C-D) = 20 + 30 + 10 + 25 = 85$$

$$\text{wt}(A-B-D-C) = 20 + 34 + 10 + 42 > 85$$

So for all path we will find minimum of all.

Code

```
int n=4;
int dist[16][16] = {
    {0, 20, 42, 25},
    {20, 0, 30, 34},
    {42, 30, 0, 10},
    {25, 34, 10, 0}};
int dp[16][4];
```

// if all cities have been visited
int visited_all = (1 << n) - 1; // (2^n - 1)

```
int tsp(int mask, int pos) {
    if (mask == VISITED_ALL) {
        return dist[pos][0];
    }
    if (dp[mask][pos] != -1) {
        return dp[mask][pos];
    }
    int ans = INT_MAX;
```


// Try to goto an unvisited city
for (int city = 0; city < n; city++) {

if ((mask & (1 << city)) == 0) {

int newAns = dist[pos][city]
+ tsp(mask | (1 << city),
city);

ans = min(ans, newAns);
// we will return min
of all paths

dp[mask][pos] =
return ans; }

int main () {

cout << tsp(1, 0) << endl;

we are starting
from A

return 0; }

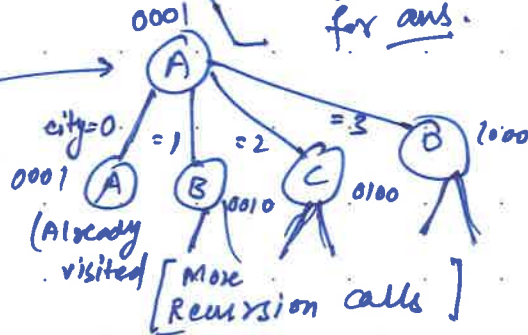
for (0 → 1 < n) {
for (0 → n) {
dp[i][j] = -1; } }

For ex:-
current
mask = 0103
1 < 3 = 0100 &
city 0100

we want
to check
which city
is visited
or not

0001 (A)
0010 (B)

0010
0001 OR
0011
we will
update
and pass
it in
recursion
for ans.



0 0 0 0
:
:
:
1 1 1 1

Distinct
States
in DP

N values
0, 1, 2, 3 → N
in position

$2^N - 1$

int tsp (int mask, int pos) {

dp[2^N][N] ∴ $O(2^N \cdot N)$

NOTE:- You can estimate the size of DP
grid or array by seeing no.
of variables in argument of funcn
changing. [Affecting the recursion's
calls]

Holiday Accomodation (# spoj.com)

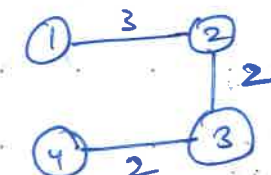
Takeaway:- N nodes & N-1 edges are there
so this graph is a tree (weighted)

∴ No cycle

I/P:-

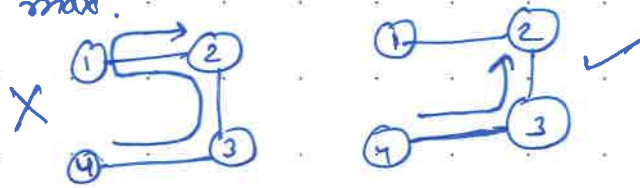
4 → No. of travellers

1 2 3
2 3 2
4 3 2



If we consider travel paths
1 - 2 → 14 ⇒ 18 ← O/P.
2 - 3 → 4

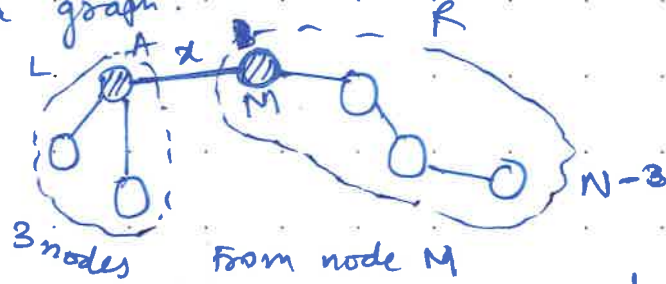
and Travellers will take shortest path but max.



We will use concept of pigeon hole.

Consider a graph

Total nodes = N



and $R = N - 3$

From node M we call DFS on L part to get count of nodes in that region.

Edge (A-M)

Contribution to the max distance travelled by travellers.

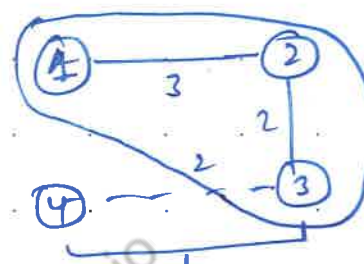
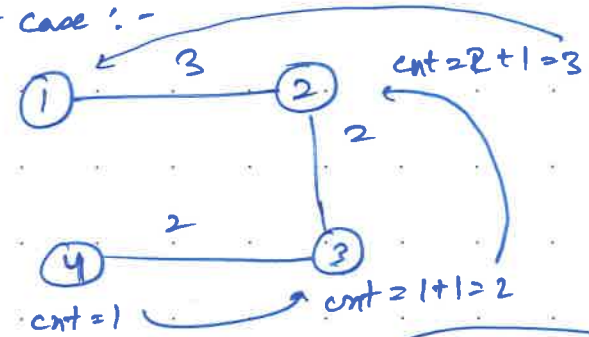
= Since L part has lesser nodes so acc. to pigeonhole only 3 travellers can exchange their homes with other 3 on R part.

$$\therefore \min(3, N-3) \times (x) \times 2$$

Two travellers exchange homes (Two times travelling)

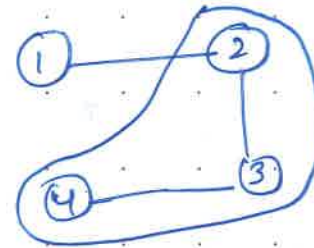
$\therefore$ We will do this for each edge and max contribution by each edge.

Our test case :-

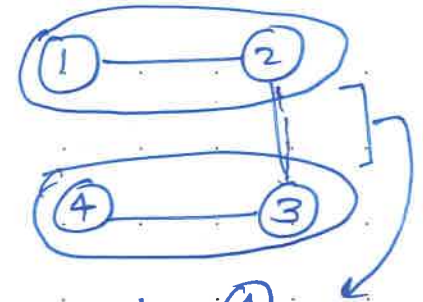


If we consider this edge then ④ can travel to ① / ② / ③ and vice versa.

$$\begin{aligned} \text{So max contribution by this edge} &= \min(1, 3) \times 2 \times 2 \\ &= 4 \end{aligned}$$



$$\begin{aligned} \min(1, 3) \times 2 \times 3 \\ = 1 \times 2 \times 3 = 6 \end{aligned}$$



③ or ④ can make travel to ① or ② vice versa.

$$\begin{aligned} \therefore \min(2, 2) \times 2 \times 2 \\ = 8 \end{aligned}$$

$$\begin{aligned} \therefore \text{Total Contribution} &= 6 + 4 + 8 \\ &= 18 \end{aligned}$$

Code

```
class Graph {
```

```
int V;
```

```
list<pair<int, int>> *l;
```

```
public:
```

```
Graph(int V) {
```

```
V = V;
```

```
l = new list<pair<int, int>>[V];
```

```
void addEdge(int u, int v, int cost) {
```

```
l[u].push-back({v, cost});
```

```
l[v].push-back({u, cost});
```

```
int dfs() {
```

```
bool *visited = new bool[V];
```

```
bool *count = new int[V];
```

```
for (int i = 0; i < V; i++) {
```

```
visited[i] = false;
```

```
count[i] = 0;
```

```
int ans = 0;
```

```
dfsHelper(0, visited, count, ans);
```

```
return ans; } // But we need this
```

```
int dfsHelper(int node, bool *visited, int
```

```
*count, int &ans) {
```

```
// mark the node visited
```

```
visited[node] = true;
```

```
count[node] = 1;
```

[weight
undirected
graph
built]

```
for (auto nbr_pair : l[node]) {
```

```
int nbr = nbr_pair.first;
```

```
int wt = nbr_pair.second;
```

```
if (!visited[nbr]) {
```

```
count[node] += dfsHelper(nbr, visited, count,
```

```
int nx = count[nbr];
```

```
int ny = V - nx;
```

```
ans += 2 *
```

```
min(nx, ny)
```

```
* wt;
```

```
// Update the
```

```
max contribution
```

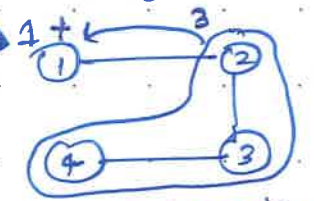
```
of this edge to
```

```
the ans.
```

Count
of
nbr
i.e. 2

Count[2] = 3

No. of
nodes
on right
side
of current
edge 1-2

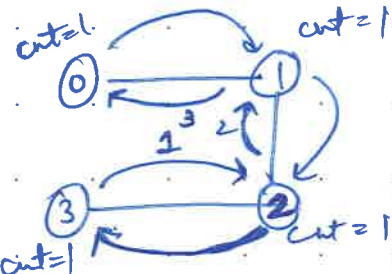


we call recursion
which returns
count of nbr.

Count of left
side of current
edge 1-2

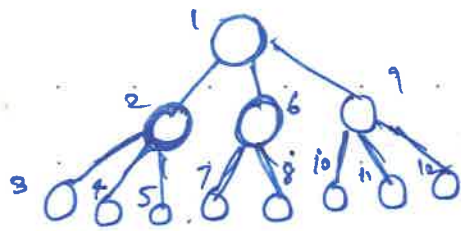
```
// just before leaving the node parent
```

```
return count[node];
```



return
count of
this node
when returning
to parent.
recursion

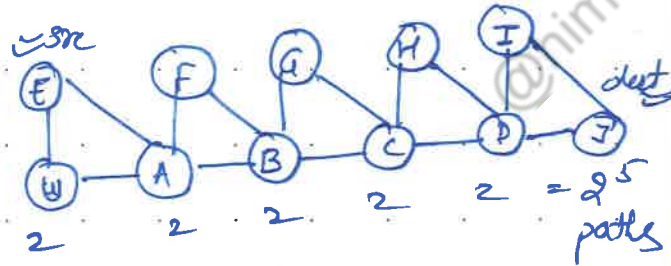
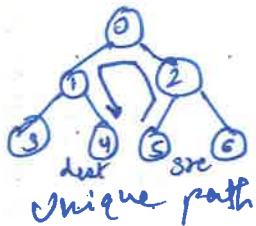
"Let food be thy medicine and medicine be thy food."



So the dfs func.
we defined traverses
in post order manner.
& For ex :-
if ② is parent
then ② will get
count of ③ + ④ + ⑤
and ② will return
the count to ①.
and so on...

NOTE :-

Dfs is not used to find shortest distance
in graph but only in Trees because we have
unique paths b/w nodes in Tree but we can
have exponential No. of unique paths b/w 2 nodes
in a graph.



→ Dfs will run in Exponential Time
→ BFS will run in Polynomial Time.
∴ We use BFS for finding shortest distance on
graph.

**GET HEALTH INSURANCE
WITHOUT MEDICAL
CHECK-UPS, AT ANY AGE!**



care freedom



Automatic Recharge
of Sum Insured



Short 2 year wait period
to cover Pre-existing Ailments



Covers insulin
dependent diabetes



Covers mental issues
like Depression, Autism &
Anxiety disorder



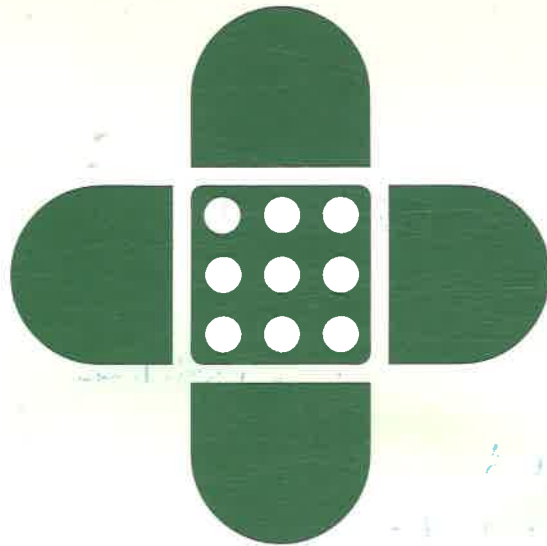
Annual Health Check-up
for all Insured members



Covers individuals with
multiple morbidities
(Hypertension, Lipid &
Obesity (BMI <= 37))

Conditions Apply

COMPREHENSIVE PERSONAL ACCIDENT COVER



Double SI payable due to Accidental Death in common carrier

3 Times benefit in case of Permanent Total Disability due to Accident

Double benefit in case of Permanent Partial Disability due to Accident

Fractures & Burns Cover

Child Education Cover

Reconstructive Surgery

Conditions Apply

Benefit may vary as per plan opted. Considering PTD & PPD & common carrier mishap optional benefit.

* Word Ladder [BFS, set] # Mostly Asked Interview

I/P:- beginword = "hit"
endword = "cog"

wordlist = ["hot", "dot", "dog", "lot", "log", "cog"]

• Shortest transformation seq.

hit → hot → dot → dog → cog

O/P:- 5

Algo

→ Set

hot	dot	dog
lot	log	cog

we maintain two variables

- 1) depth of BFS traversal $\Rightarrow$ depth = 0
- 2) level-size of current depth $\Rightarrow$ lsize = 0

Queue

hit

- 1) check if this string == endword first, and push (beginword) first and lsize = 1
- 2) Increment depth value by 1 (depth = 1) and pop front element (i.e hit) and store in string variable
- 3) hit
 we run loop (a → z) and check for each char if exist

- hit → Now check for h ? t
 > hot is in set so push it in queue.

- hit → check for hi ?

Now ~~the~~ the only word at this ~~depth~~ depth is completely traversed.

∴ depth++ ⇒ depth = 2

hit | hot | lsize = 2

Perform same steps unless q.empty() == ~~false~~

← 1 → ← 2 → ← 3 → ← 4 → depth ←

hit	hot	dot	lit	dog	log
-----	-----	-----	-----	-----	-----

depth = 0 1 2 3 4 5
 lsize = 1 0 1 0 1 2

Time complexity

$$= O(26 * N * N * W)$$

char checked (constant) for string compare length of word No. of words

$$= O(N^2 * W)$$

Code

```
int ladderLength(string beginWord, string endWord,
vector<string> &wordList)
```

```
unordered_set<string> myset;
bool isPresent = false; // checks if endWord
                        is present in wordList
```

```
for(auto word : wordList)
```

```
if (endWord == word) {
    isPresent = true;
}
```

```
myset.insert(word);
```

```
if (isPresent == false) { // endWord not present
    return 0;
}
```

```
Queue<string> q;
```

```
q.push(beginWord);
```

```
int depth = 0;
```

```
while (!q.empty()) { // increase depth
```

```
    depth++;
    int lsize = q.size(); // no of elements at a level
```

```
    while (lsize-- > 0) {
```

```
        string curr = q.front();
```

```
        q.pop(); // remove curr element
```

```
        // check for all possible depth pos in curr
```

```
        for (int i = 0; i < curr.length(); i++) {
```

```
            string temp = curr;
```



```

for (char c = 'a'; c <= 'z'; c++) { // Try all possible char for current pos^n
    temp[i] = c;
    if (curr == temp) { // if hit hit 'h' comes again; skip
        continue; }
    if (temp == endWord) { // if the temp comes out to endWord then increase the depth & return
        return depth + 1; }
    if (myset.find(temp) != myset.end()) { // else push if the element is in set,
        q.push(temp);
        myset.erase(temp);
    }
}
}
}
}

```

Apple ②

THROUSTES (codechef) - Tree Question
(Applying D)

THOUSES (Codechef) - 1822

(Applying DFS on tree - (like we do in graph when only edges $(u \rightarrow v)$ are given))

```
ll dfs(ll src, umap<ll, list<ll>> tree -  
    &l, umap<ll, ll> &parent) {  
    ll i = 0;  
    ll i = 0; // stores no. of children for current src.  
    vector<ll> vec;  
    ll sum = 0; // store sum of what get from each child
```

```

for (auto child : x[src] / 2)
    if (parent[src] != child) {
        it++; // increasing i for valid cases
        parent[child] = src; // (invalid case = parent)
        // Update parent
        recursive [ ll item = dfs(child, l, parent); // call dfs on child
        vec.push_back(item);
    }
}
sort(vec.begin(), vec.end());
for (auto x : vec) {
    sum += (i * x); // Allotting largest value to node who has least sum in its subbranch.
    i--;
}
vec.clear();
return sum + 1; // current root
}

```

```

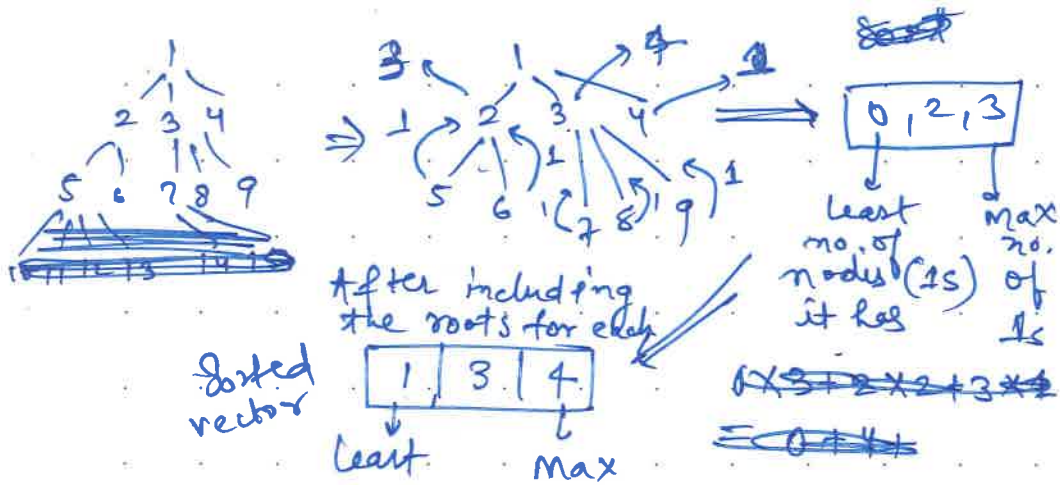
11 solve(1) {
12     if (n, x
13         n > x;
14         unmap k;
15     for (0 to n-1) {
16         u, v;
17         if (u, push back(v);
18         if (v, push back(u);
19     }
20     unmap parent;

```

```

for (auto p : l) {
    all node = p.first;
    parent[node] = node;
}
ans = dfs(1, l, parent)
% MOD;
cout << (ans * x) % MOD << endl;
}

```



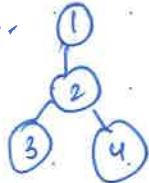
$$1 \times 3 + 3 \times 2 + 4 \times 1 = 3 + 6 + 4 = 13$$

We return $(13+1)$ to ①

So ans = 14

VPATH [Codechef]

HP:-



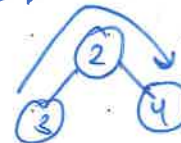
o/p:- $\{3, (2, 3), (4), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4), (1, 2, 3), (1, 2, 4), (2, 3, 4)\}$

A set S of nodes is called valid if there exists two vertices u and v . (poss. $u=v$) such that every node in S lies on the simple path from u to v .

Case 1:- Counts straight paths only.



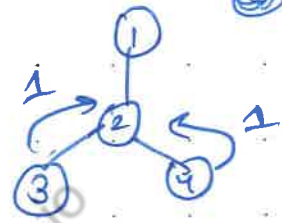
Case 2:- Curved straight paths.



$(3, 4), (2, 3, 4)$

We will return no. of possible straight paths to root, because that will be used to calculate no. of straight path from current root.

For ex:-



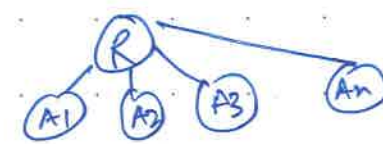
③ for leaf node only
+ case $\rightarrow \{3\}$

② for this case
 $\{2\}, \{3\}, \{1, 3\}$

NOTE:- Don't count

$\{3\}$ multiple times this will result in ~~redundant~~ subsets. So only during returning we will return all possible straight paths.

No. of possible straight subsets. we return ~~3~~, 3.



If a root has n children then no. of ~~straight~~ curved straight paths.

$$(A_1 A_2 + A_2 A_3 + A_1 A_3 + \dots)$$

recursion 1, recursion 2, recursion 3. In this case, it is inefficient, multiple times same recursion is being called. efficient

$$(A_1 A_2 + A_2 A_3 + \dots) = (A_1 + A_2 + A_3 + \dots)^2 - (A_1^2 + A_2^2 + A_3^2 + \dots)$$

Just because you're not sick doesn't mean you're healthy."

In this there will be two sub cases where we include root in curved straight path subset or not.

For ex:- (2,3,4) and (3,4)
 $\downarrow$ with root $\downarrow$ without root

$\therefore$ we will calculate $2(A_1A_2 + A_2A_3 + A_3A_4 + \dots)$

Code

```

int ans = 0; // stores final no. of subsets
int dfs (int src, unordered_map<int, list> &ll, unordered_map parent) {

```

```

    int sum = 0; // stores straight path case
    int help = 0; // stores curved straight path case,

```

```

    for (auto child : ll[src]) {

```

```

        if (parent[src] != child) {

```

```

            parent[child] = src;
            int item = dfs (child, ll, parent); // It will return

```

```

            help += (item * item) % MOD;

```

```

            help = (help + MOD) % MOD;

```

```

            sum += item % MOD;

```

```

            sum = (sum + MOD) % MOD; }

```

```

        ans += (sum * sum - help) % MOD; // curved st. path case
        ans = (ans + MOD) % MOD;

```

```

        ans += (sum + 1) % MOD; // possible subsets from
        ans = (ans + MOD) % MOD; // all children in this sub branch

```

```

        return (2 * sum + 1) % MOD; // cumulative (all)
        // subsets forming straight path

```

Hashing / Hash table

Purpose: To support insertion, deletion & search in average-case constant time.

- Assumption: Order of elements is irrelevant.
- This D.S. not useful if you want to maintain & retrieve some kind of an order of the elements.

• Hash function

$\Rightarrow \text{Hash}[\text{"string key"}] \Rightarrow \text{integer value}$

Key components

- Hash Function (Good)
- Hash Table (that supports insert, delete, search & delete)
- Collision handling scheme. (for keys collisions)

⊛ Hash table uses hash function to compute indexes for a key.

⊛ It provides very fast access time. While there can be collisions, if we choose a very good hash func, this chance is almost zero.

⊛ On average, T.C. is a constant $O(1)$ access time.

here we have taken sum+1 because then redundant nodes will be counted

me no. of straight path subsets from bottom to child in this sub branch

To avoid overflow

This is called Amortized Time Complexity.

- Unordered-map (STL) D.S. implements all these hash functions.

□ [Hashing described in other DSA N.B.]

- Hash Table is an array of fixed size (Table size)
- Array elements indexed by a key, which is mapped to an array index (0 to Table size-1) (Implementation using dynamic array)
- mapping (hash function) h from key to index
e.g. $R(\text{"John"}) = 2$

Operations

Insert - $T[h(\text{key})] = \text{value};$
delete - $T[h(\text{key})] = \text{NULL};$
search - return $T[h(\text{key})];$

Hash Function

- ~~Collision~~ A "good" hash func should have the properties:
 - ➔ Reduced chance of collision - Distribute keys uniformly over table.

➔ Should be fast to compute

Hash Func Example

- Simple Hash Func

$$h(\text{key}) = \text{key} \% \text{Table size}$$

⇒ But if table size = 100 and keys are All multiples of 10

➔ Better if Table size is a prime no.

How to handle strings?

⇒ Add up char ASCII values (0-255) to produce integer keys

$$\text{"abcd"} = 97 + 98 + 99 + 100 = 394$$

$$h(\text{"abcd"}) = 394 \% \text{Table size}$$

Potential Problems:-

- Anagrams will map to the same index. [$h(\text{"abcd"}) = h(\text{"dbac"})$]
- Small strings may not use all of table
- Time $\propto$ length of string

Better Approach

$$h(s) = \left[\sum_{i=0}^{L-1} s[L-i-1] * 37^i \right] \% \text{Table size}$$

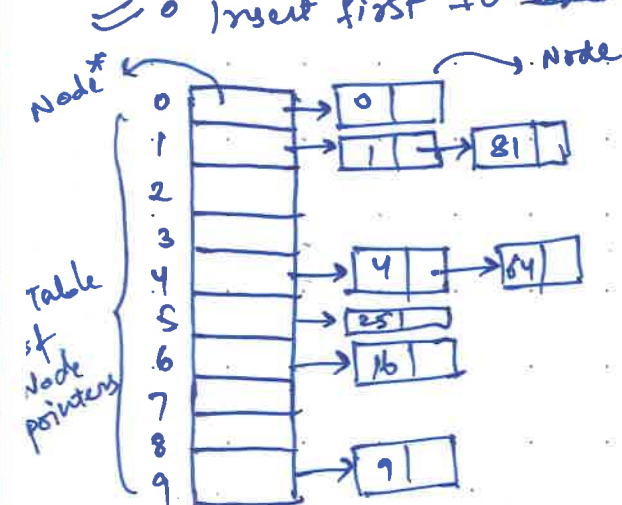
String

Collision Handling Schemes

- 1) Open Hashing - Separate Chaining
- 2) Closed Hashing - Open Addressing
 - Linear Probing
 - Quadratic Probing
- 3) Double Hashing

Separate Chaining

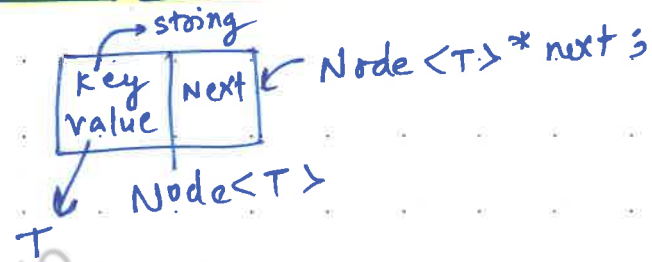
- Implemented using LL.
- Key k is stored in list at $T[h(k)]$
- Fig. Tablesize = 10
 - $h(k) = k \bmod 10$
 - Insert first 10 ~~even~~ perfect squares.



hash fn
 $= \text{No.} \% 10$
 $\Rightarrow \text{idx}$

- Only Time will be taken to traverse the corresponding LL after find index so we will set constraint of sized (L) < 20 . $\sim O(1)$ constant traversal time

Class Implementation (Hash Table)



```
template < typename T >
```

```
class Node {
```

```
public:
```

```
    string key;
```

```
    T value;
```

```
    Node<T> * next;
```

```
    Node(string key, T val) {
```

```
        this->key = key;
```

```
        value = val;
```

```
        next = NULL;
```

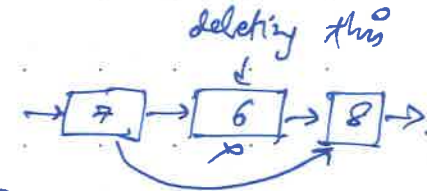
```
    } Node() {
```

```
        if (next != NULL) {
```

```
            delete next;
```

```
        }
```

```
    }
```



it will start a new chain & it will delete next nodes


```
template <typename T>
```

```
class HashTable {
```

```
Node <T> ** table;
int current-size;
int table-size;
```

→ declared ~~dynamic~~ array of double pointer of type Node**

```
public:
```

```
HashTable (int sz=27) {
    table-size = sz;
    table = new Node <T> *[table-size];
    current-size = 0;
    for (int i=0; i<table-size; i++) {
```

assignment double pointer pointing at start of a dynamic array of pointers.



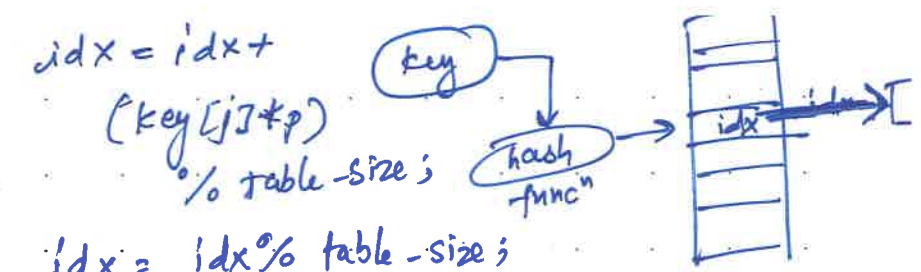
// The bucket will be assigned ~~initialized~~ with null so that garbage values sweep out and null bucket will tell us that it is not pointing to any list yet.

```
}
void insert (string key, T value) {
```

```
int hashFn (string key) {
```

```
int idx = 0;
int p = 1;
for (int j=0; j<key.length(); j++) {
```

bit man
 $b + a \times 27 + c \times 27^2 + \dots$



```
idx = idx % table-size;
```

~~idx = (p * 27) % table-size;~~ } Q Why taking mod everytime

```
void insert (string key, T value) {
    int idx = hashFn (key);
```

```
Node <T> * n = new Node <T> (key, val);
```

// Insert at head of LL with id = idx

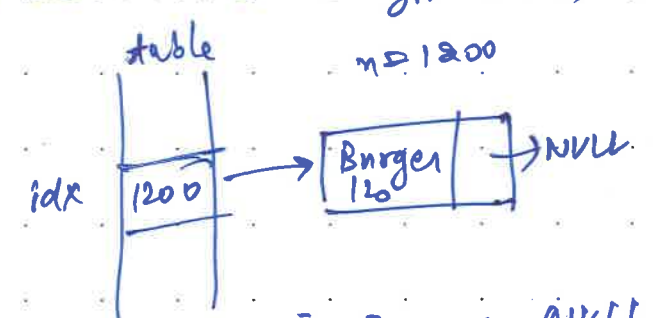
```
n->next = table[idx];
```

```
table[idx] = n;
```

```
current-size++;
```

// rehash (Code for it given ahead) % m

A. 27^{13} (big)
 ↓
 it may overflow
 ↓
 -ve idx
 ↓
 seg fault
 using $(a+b+c) \% m = (a \% m + b \% m + c \% m) \% m$



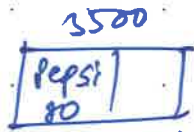
At start table[idx] was NULL so we set n->next = table[idx]

"Those who think they have no time for exercise will sooner or later have to find time for illness."

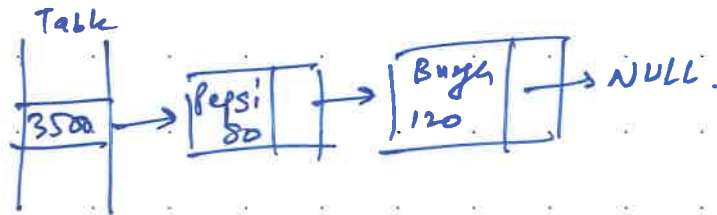
i.e. $n \rightarrow \text{next} = \text{NULL}$

and $\text{table}[\text{idx}] = n$;
that means $\text{table}[\text{idx}]$ is pointing to node.

If we add another node



then we are actually performing head insertion at LL.



void print() {

```
for (int i = 0; i < table; i++) {  
    cout << "Bucket " << i << " → ";  
    Node <T> *temp = table[i];  
    while (temp != NULL) {  
        cout << temp->key << " → ";  
        temp = temp->next;  
    }  
    cout << endl;  
}
```

[the above complete code will be a part of "hashtable.h" file.]

filename → "hashtable.cpp".

#include "hashtable.h"

User defined
Libraries
are included
in ""

int main() {

HashTable<int> price-menu;

price-menu.insert("Burger", 120);

price-menu.insert("Pepsi", 20);

price-menu.insert("Pizza", 150);

price-menu.insert("Noodles", 25);

price-menu.insert("Coke", 40);

price-menu.print();

}

Load Factor
 $= \frac{5}{7}$

O/P

Bucket 0 →

Bucket 1 →

Bucket 2 → Noodles →

Bucket 3 → Burger →

Bucket 4 → Coke →
Pepsi →

Bucket 5 → Pizza →

Bucket 6 →

Load Factor &
Rehashing-

Insertion (How hashing
works)

for a key (K) - value (V) pair
insertion,

1) K is converted into a small int (hash
code) using a hash function,

2) Hash code is used to find idx & entire
" at that idx (separate chaining)

is first searched for the presence of k .

3. If found $\rightarrow$ updated
else $\rightarrow$ new node of $\{k, v\}$ added in List.

Complexity & Load Factor

- For large no. of entries into map and each key can be found & outputted in constant time - $O(1)$.
- For the worst case all entries are at the same index. So T.C. is $O(n)$. Good hash func distributes entries very well.
- on an average, if there n entries and m is the size of the array, so there would be $\left(\frac{n}{m}\right)$ entries on each index.

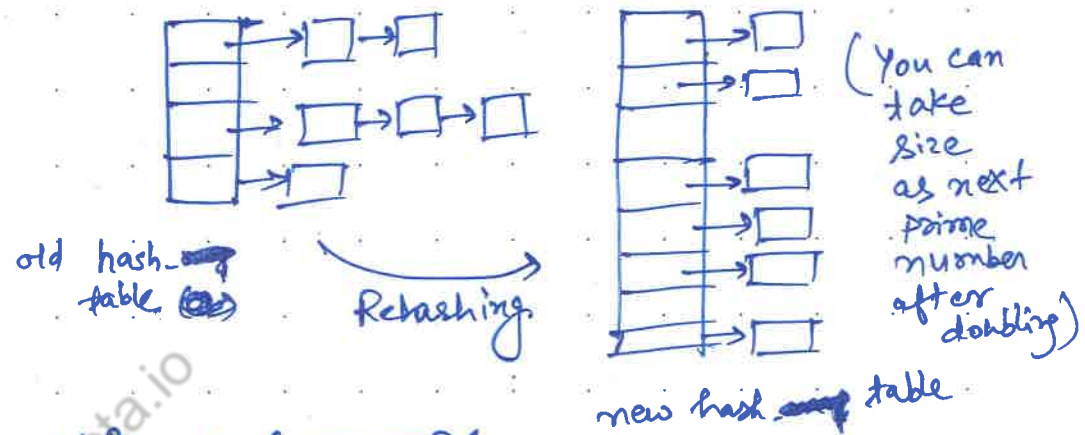
$$\left(\frac{n}{m}\right) = \text{Load Factor}$$

, represents load on our map.

Rehashing

when load factor increases to more than its pre-defined value (default load factor = 0.75), the complexity increases. So to overcome this, the size of array is increased (doubled) & the values are

hashed again & stored in the new double sized array to maintain a low load factor and low complexity.



~~void insert~~

~~rehash (continuation)~~

~~load factor~~

$\Rightarrow$ in class HashTable, in private section,

void rehash () {

Node <T> ** oldTable = table; $\rightarrow$ int oldTableSize = TableSize;

tableSize = 2 * tableSize;

table = new Node <T> * [tableSize];

for (int i = 0; i < tableSize; i++) {
table[i] = NULL;

currentSize = 0;

// Shift the elements from old to new
for (i = 0; i < oldTableSize; i++) {

```
Node<T>* temp = oldTable[i];
```

```
while (temp != NULL) {
    insert(temp->key, temp->value);
    temp = temp->next;
}
```

```
if (oldTable[i] != NULL) {
```

```
    delete oldTable[i];
```

```
}
```

// as we know each node is dynamically created so we also have to call destructor through "delete".

```
delete [] oldTable;
```

```
}
```

NOTE:-

Now the new hash table made will have lesser no. of collisions as compared to previous one.

```
T* search (string key) {
```

```
    int idx = hashfn(key);
```

```
    Node<T>* temp = table[idx];
```

```
    while (temp != NULL) {
```

```
        if (temp->key == key) {
```

```
            return temp->value;
```

```
            temp = temp->next;
        }
```

```
    return NULL;
```

```
}
```

In int main,

```
int *price = price-menu.
```

```
search("nodul");
```

if (price == NULL) { cout << "not found"; } else { *price; }

key	
value	

↑
temp

search fn. will return pointer to

the value of searched item

Easy Access [] operator

```
price-menu["dosa"] = 60; // Insert
```

```
price-menu["dosa"] = 90; // Update
```

```
cout << price-menu["dosa"];
```

Code:-

```
T & operator[](string key) {
```

```
    T * f = search(key);
```

```
    if (f == NULL) {
```

```
        // insert key, value (garbage)
```

```
        in the hashmap
```

```
        T garbage;
```

```
        insert(key, garbage);
```

```
        f = search(key);
```

```
    return *f; }
```

key	
value	

we will pass by reference so that it doesn't send copy & we can't update it

Dosa	
Garbage	

f →

• (*itr).first / itr->first

Map() in STL

↓
associative containers

map() is implemented using self balancing Binary Search Tree such as AVL, Red & Black Tree.

T.C. (in all cases) - $O(\log n)$

```
map<string, int> m;
```

```
m.insert(make_pair("mango", 100));
```

```
pair<string, int> p;
```

```
p.first = "Apple";
```

```
p.second = 120;
```

```
m.insert(p);
```

```
m["Banana"] = 20;
```

```
string fruit = "Banana";
```

```
auto if (m.find(fruit) != m.end()) {  
    m[fruit]; } else { not present; }
```

```
m["Banana"] = 30; → update + overwrite
```

```
if (m.count(fruit) > 0 → it passes integer  
    m[fruit]; } unlike find()
```

```
else { not present; }
```

```
for (auto itr = m.begin(); itr != m.end(); itr++) {  
    cout << itr->first << " " << itr->second << endl; }
```

```
for (auto x : m) {
```

```
    cout << x.first << " " << x.second; }
```

```
m.erase(fruit);
```

auto / map<string, int> :: iterator

Unordered_Map()

implemented using hash table.

T.C. → $O(1)$ for search, delete, insert.

In unordered-map, ~~ordering~~ elements are not present in order. (Random order)

Applying ~~auto~~ on custom-class

```
class Student {
```

```
public:
```

```
    string firstname;
```

```
    string lastname;
```

```
    string rollno;
```

```
    Student(string f, string l, string no) {
```

```
        firstname = f;
```

```
        lastname = l;
```

```
        rollno = no; }
```

```
    bool operator==(const Student &s) {  
        return rollno == s.rollno; }
```

```
};
```

func for comparing two student class objects

"If you don't take care of your body, where are you going to live?"

class HashFn {
 public:
 size_t operator()(const Student &s) const {
 return s.firstname.length() + s.lastname.length();
 }
};

int main() {

unordered_map<Student, int, HashFn> student_map;

Student s1("Prateek", "Naray", "010");

s2("Lahul", "Kumar", "023");

s3("Lahul", "Kumar", "120");

student_map[s1] = 100;
 student_map[s2] = 120;
 student_map[s3] = 11;

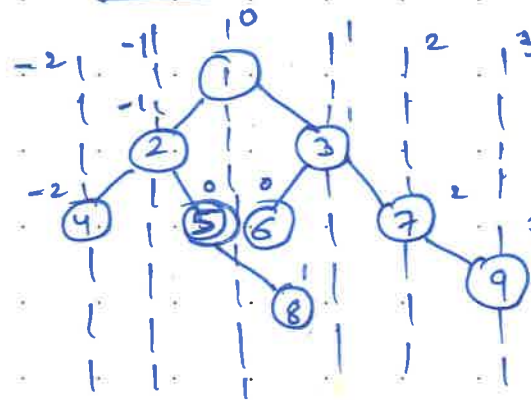
for(auto &s : student_map) {
 s.firstname << " " << s.lastname << " " << s.second;
}

NOTE:-

We can't use custom objects like pair<>, class objects etc. in unordered_map, unordered_set as key. Because then we also have to define hash func & comparison function class function.

At these times, we use map or set because there many times comparison is done on the basis of first instance or we can just define class.

Vertical order Print # Trees # Hashing



O/P:-

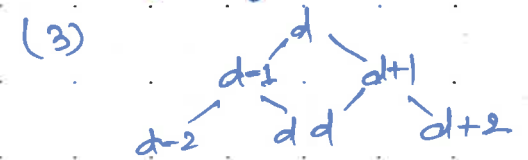
4
 2
 1 5 6
 3 8
 7
 9

Hashing done!

key	Value
-2	4
-1	2
0	1 5 6
1	3 8
2	7
3	9
int	list<int> or vector<int>

Algo

- we take root at 'd' distance
- if we go left we do (d-1) & if we go right then (d+1).



code

class node {

};

void verticalOrderPrint(node* root, int d, map<int, vector<int>> &m) {
 if (root == NULL) return;
 m[d].push_back(root->data);


```

verticalOrderPrint (root → left, d-1, m);
verticalOrderPrint (root → right, d+1, m);
return; }

```

```

int main () {
    // tree builder //
    map<int, vector<int>> m;
    int d=0;
    verticalOrderPrint (root, d, m);
    for (auto p: m) {
        cout << p.first << "→";
        for (int x: p.second) {
            cout << x << ", ";
        }
        cout << endl; }
}

```

0/1 knapsack

*** * NOTE :- * ***

We initialize row 1 & column 1 as 0 initially: because for 0 item obviously we won't get any bag filled & technically in 0-1/0-n knapsack we don't need the sum of wt. to be equal to knapsack capacity exactly everytime. So in case

Ex: w=6 & only 1 item (1, 4) is there.
 If we include 1kg item then we check for 2kg item but since no item is present we can return profit by 4 only.

0/1 Knapsack [Dynamic Programming]

W=8
 N=4
 Profit = {1, 2, 5, 6}
 Wt = {2, 3, 4, 5}

		W									
		0	1	2	3	4	5	6	7	8	
Pi	Wti	0	0	0	0	0	0	0	0	0	
1	2	1	0	0	1	1	1	1	1	1	
2	3	2	0	0	1	2	2	3	3	3	
5	4	3	0	0	1	2	5	5	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

$VE[i][w]$ = max profit so far where i = No. of objects
 w = weight of capacity.

Algo

- For $i=0$, that means no objects is taken at this moment.
- For $i=1$, 1st item we will consider. since $wt[0] = 2$ so the wt. of the capacity (w) = 2 and at $V[1][2] = 1$ (Profit from 1st item).
- For $i=2$, we will consider 1st & 2nd item. Since weight ($2 < w \leq 8$) doesn't exist for this case so $V[1][3]$ to $V[1][8] = 1$ & $V[1][1] = 0$ (max from prev. value in the row).

3) for $i=2$; both $\{1,2\}$ & $\{2,3\}$ items will be considered to find values of specific weight of capacity.

if we take only $\{2,3\}$ item then at $V[2][3] = 2$ and for $\{1,2\}$ $V[2][2] = 1$.

if we consider both item together then $wt = 2+3 = 5$, $P = 1+2 = 3$
 $\therefore V[2][5] = 3$ & now we fill $V[2][6]$ to $V[2][8] = 3$.

4) for $i=3$; $\{1,2\}$, $\{2,3\}$ & $\{5,4\}$

if $\{1,2\} \Rightarrow V[3][2] = 1$

if $\{2,3\} \Rightarrow V[3][3] = 2$

if $\{5,4\} \Rightarrow V[3][4] = 5$

if $\{1,2\}, \{2,3\} \Rightarrow$ Profit = 3 which is less than previous value in the same row
 $\therefore$ Profit = 5
 $\therefore V[3][5] = 5$

if $\{1,2\}, \{5,4\} \Rightarrow V[3][6] = 6 \leftarrow$ New max

if $\{2,3\}, \{5,4\} \Rightarrow V[3][7] = 7 \leftarrow$ global max

$\{1,2\}, \{2,3\}, \{5,4\} \Rightarrow wt = 1+2+3 = 6$
 $\therefore$ Not Applicable.

Recurrence DP Formula

$$V[i][w] = \max \{ V[i-1][w], V[i-1][w-wt[i]] + P[i] \}$$

For ex:-

$$V[4][1] = \max \{ V[3][1], V[3][1-5] + 6 \}$$

$$= \max \{ 0, V[3][-4] + 6 \}$$

$$V[4][1] = 0 \leftarrow \text{Not exists}$$

So till what "w" we will get such not existing wt. of the capacity (w) value?

Till $w=5$, so till this, we assign same value to each cell that of above it. i.e. $V[4][2] = V[3][2]$,
 $V[4][3] = V[3][3]$ and $V[4][4] = V[3][4]$.

Now,

$$V[4][5] = \max \{ V[3][5], V[3][5-5] + 6 \}$$

$$= \max \{ 5, 0 + 6 \} = \max \{ 5, 6 \}$$

$$= 6$$

$$V[4][8] = \max \{ V[3][8], V[3][8-5] + 6 \}$$

$$= \max \{ 7, 2 + 6 \} = 8$$

$\{x_1, x_2, x_3, x_4\} \Rightarrow 4$ items
 0 1 0 1

we see max ~~val~~ profit we obtained
 = 8.

$\Rightarrow$ Since 8 only lies in 4th row that means because of 4th item we got this max profit. So x_4 will be there in capacity bag.

Now, $\Rightarrow$ what's the profit from 4th item.
 = 6, $\therefore 8 - 6 = 2$

$\Rightarrow$ From 2nd row we got 2 as ~~max~~ profit so include x_2 but not x_3 .
 and now $2 - 2 = 0$
 $\downarrow$
 profit of 2nd item

$\Rightarrow$ 0 profit we got from 0th row.
 so we won't include x_1 .

$\therefore \{x_2, x_4\} \Rightarrow$ final items

resulting max profits

 In knapsack we find out max value subset of $val[]$ such that sum of the weights of this subset is smaller than or equal to W .
 $\text{Total wt} \leq W \rightarrow$ knapsack cap

Code

$\rightarrow$ (Cap)

int knapsack(int W, int wt[], int Profit[], int n)

int i, w;

int k[n+1][W+1];

$\rightarrow$ loop for no. of items

for (i=0; i<=n; i++)
 for (w=0; w<=W; w++) {

$\rightarrow$ current wt. of capacity

if (i==0 || w==0)

k[i][w] = 0;

if wt of item is less or equal to current capacity

else if (wt[i-1] <= w)

$\rightarrow$ exclude

k[i][w] = max(k[i-1][w], Profit[i-1] + k[i-1][w - wt[i-1]]);

$\downarrow$
 include

if capacity is not enough then only exclude

else {

k[i][w] = k[i-1][w];

if

wt of item comes out to be more than current wt. (space) of capacity then

store previous row same column cell's value.

return k[n][W];

$\rightarrow$ max profit

 There is no compulsion that total weight for max profit should be W exactly

Knapsack Capacity

"From the bitterness of disease man learns the sweetness of health."

Watching CPL (codechef) ~~# DP~~ DP
(code:-WIPPL) ~~# 0-1 Knapsack~~ 0-1 Knapsack

I/O:-

O/P:-

① 2
8 38 → height of each box. 7
7 8 19 7 8 7 10 20
② 4 5 → height of each box. 2
2 10 4 9 → height[]

~~Box Array~~

Algo

- Sort all boxes in non-decreasing order.
- Run a knapsack of the form $dp[i][j]$ min. height greater than equal to j that can be made using only the i boxes.
- 2 towers can be made using the i boxes if $prefix[i] - dp[i][K] \geq K$
That means with current sum of i boxes we can accommodate one more ~~box~~ ^{tower} making it total two towers possible.
- Find the min i and return it.

For test case 2.

ht[i]	dp	0	1	2	3	4	5
0	0	0	0	0	0	0	0
10	1	0	10	10	10	10	10
9	2	0	9	9	9	9	9
4	3	0	4	4	4	4	9
2	4	0	2	2	4	4	6

(height of the tower needed)

i , where ~~height of tower~~
 i = no. of boxes we can include from 0 to i (inclusive).

- For $i=0$ (0 box), no height could be reached. $\therefore dp[i][j]$ (for all j) = 0.
For $j=0$, no box is needed $\therefore dp[i][0] = 0$ ($\forall i$).
- For $i=1$, $ht[1]=10$.
So all required tower height can be build with this box so min. height = 10.
- For $i=2$, $ht[2]=9$.
This box can accommodate all heights j and less than the previous row every cells values $\therefore dp[2][1 \rightarrow 5] = 9$.
- For $i=3$, $ht[3]=4$.
This box can't accommodate height of tower = 5. In this case we require 2 boxes (if we include box of height 4).

on its own

$5 - 4 = 1 \rightarrow$ 1 more height required to reach 5 height of tower.
So we look up previous row for $j = 1$ value. Since $dp[3-1][1] = 9$.

Two cases:-

① If we include box of height 4.
 $\Rightarrow (4, 9)$ [2 boxes]

Total height = 13

② If we ~~include~~ exclude box of height 4 then we include the next greater height box i.e. ~~box of height 4~~
 $dp[2][1] = 9$.

$\Rightarrow (9)$ [1 box]

$$dp[3][5] = \min(dp[3-1][5], dp[3-1][5 - ht[3]] + ht[3])$$

$$= \min(9, 9 + 4)$$

$$dp[3][5] = \min(9, 13) = 9$$

* for $j = 4$ ~~at~~ $1 \leq j \leq 4$ ~~at~~ This box can accommodate. $\therefore dp[3][1 \rightarrow 4] = 4$

5) for $i = 4$, ~~a~~ $ht[4] = 2$.

This box can't accommodate height of tower of from [3, 5] on its own.

From above observation,

dp formula $\triangleright$

$$dp[i][j] = \min(dp[i-1][j], dp[i-1][j - ht[i]] + ht[i])$$

excluding ~~ht[i]~~ box
This previous row same col. cell will store min ht / val. cumulated till now.

including $ht[i]$ box

• After getting ~~dp[3][5]~~ complete dp table.

• Make prefix array of $ht[]$. ($k = 5$)

prefix[]

10	19	23	25
1	2	3	4

①

$$prefix[1] - dp[1][k]$$

$$\Rightarrow 10 - 10$$

$$\Rightarrow 0 < K$$

②

$$19 - 9 = 10 \geq K$$

$\downarrow$
Therefore one max tower of height 5 can be made.

Two towers of height 5 can be made using ^{min} 2 boxes of heights 10 & 9. ✓

Code

```

ll dp[4000][4000];
void cpl(ll n, ll k, ll ht pref[j]) {
    for (0 → n) {
        for (0 → k) {
            dp[i][j] = INT_MAX;
        }
        for (0 → n) { dp[i][0] = 0; }
        for (0 → n) {
            for (1 → k) {
                if (pref[i] >= j) {
                    dp[i][j] = pref[i];
                    continue;
                }
                if (i-1 < 0 && pref[i] < j) {
                    dp[i][j] = INT_MAX;
                }
                else {
                    dp[i][j] = min(dp[i-1][j],
                                   dp[i-1][j - pref[i]]
                                   + pref[i]);
                }
            }
        }
    }
}

```

if we lookup out of dp (not valid) →

case where box height is more than current j

```

ll helper(ll n, ll pref, ll k) {
    for (0 → n) {
        if (pref[i] - dp[i][k] > k) {
            return i+1;
        }
    }
    return -1;
}

```

```

void solve() {
    ll n, k;
    cin >> n >> k;
    ll ht[n];
    for (0 → n) { cin >> ht[i]; }
    sort(ht, ht+n, greater<int>());
    ll pref[n];
    // Calculate prefix array
    cpl(n, k, ht);
    ll ans = helper(n, pref, k);
    cout << ans << endl;
}

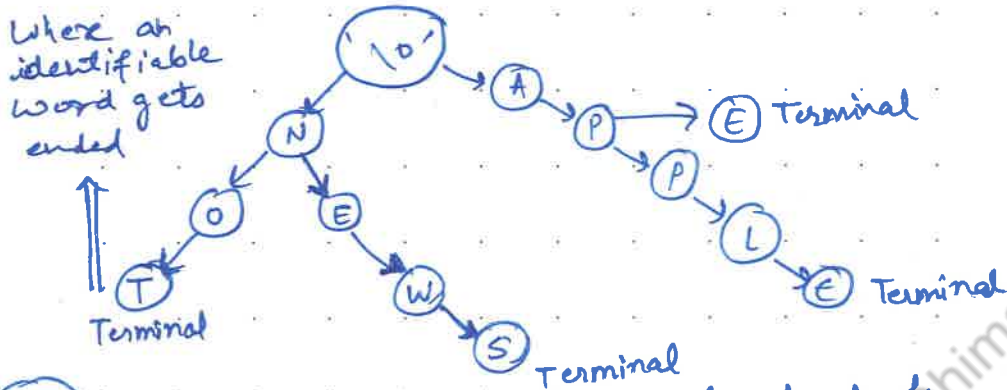
```

Tries

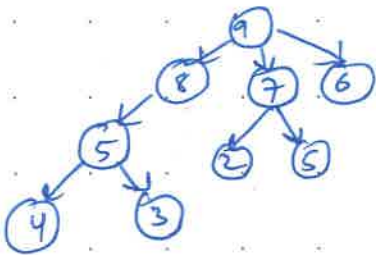
Problem: list of strings
 "No", "Not", "Neries", "Hello", "Apple" ...

Queries :- Not $\Rightarrow$ True
 New $\Rightarrow$ False

$\therefore$ Prefix Tree $\Rightarrow$ Trie can be used to find this in optimal time.



⌘ If you want to implement phonebooks.
 • For huge data (huge no. of phone nos.)
 = Use Trie data structure (because most of the phone nos. have same prefix)



For comparatively lesser data (around 1000 phone nos.)
 = use hashmap / hash table

Implementation

```

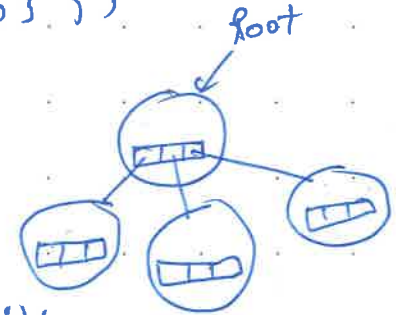
class Node {
public:
    char data;
    unordered_map<char, Node*> children;
    bool terminal;
}
    
```

```

Node(char d) {
    data = d;
    terminal = false;
}
    
```

```

class Trie {
    Node* root;
    int cnt;
public:
    Trie() {
        root = new Node('0');
        cnt = 0;
    }
    void insert(char* w) {
        Node* temp = root;
        for (int i = 0; w[i] != '\0'; i++) {
            // traverse the word char by char
            char ch = w[i];
        }
    }
}
    
```



word to be inserted

At each node we check hash map if required char reside in any of the children if not we create new child

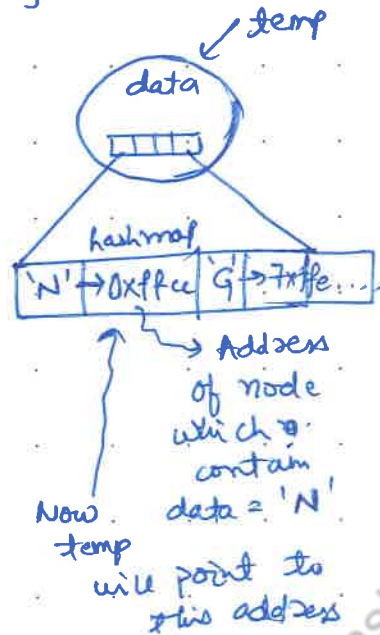
“True healthcare reform starts in your kitchen.”


```

if (temp → children.count(ch))
    temp = temp → children[ch];
else {
    Node *n = new Node(ch);
    temp → children[ch] = n;
    temp = n;
}

```

hash map of the node temp is pointing to



```

// else we created a new node and update the children hashmap of current node and now temp will point to the new node
}
}

```

```

temp → terminal = true;
// when we reach end of the word we mark data member terminal = true to mark it the end
}

```

```

bool find(char *w) {
    // String to be find

```

```

    Node *temp = root;
    for (int i=0; w[i] != '\0'; i++) {
        char ch = w[i];
        if (temp → children.count(ch) == 0) {
            return false; // If any char can't be find in hashmap
        } else {
            temp = temp → children[ch]; // if current char matches move the
        }
    }

```

```

    }
    return temp → terminal;
}
}
int main() {
    Trie t;
    char words[100] = {"a", "hello", "not", "news", "news", "apple"};

```

At last return whether the last node is terminal or not (terminal node ⇒ where word gets ended)
(if it were not to be a word then terminal remains = false)

```

    char w[10];
    for (int i=0; i<10; i++) {
        cin >> w;
        t.insert(words[i]);
    }

```

word to be searched for

```

    if (t.find(w)) {
        cout << "Present";
    } else {
        cout << "Absent";
    }

```

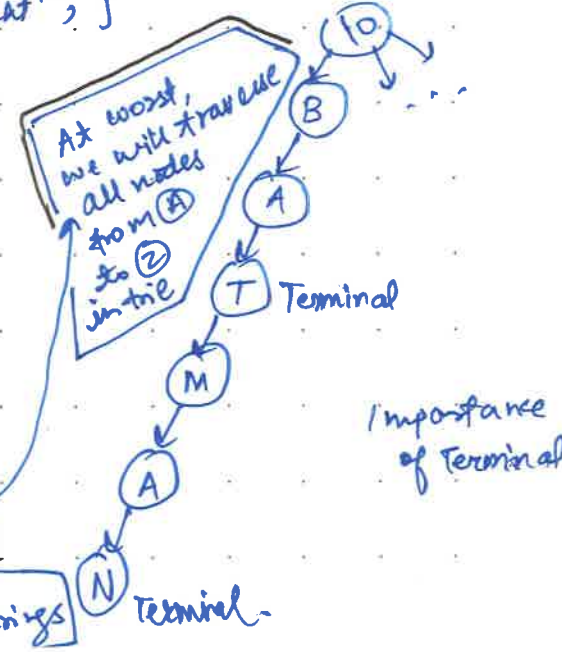
```

    return 0;
}

```

For smaller strings it takes almost $O(1)$ time to insert or find

Generally ⇒
 $T.C = O(26 * M * N)$
 where M is no. of strings, N is length of strings



Unique Prefix Array

words[] = ["cobra", "dog", "dove", "duck"]

output[] = ["c", "dog", "dov", "du"]

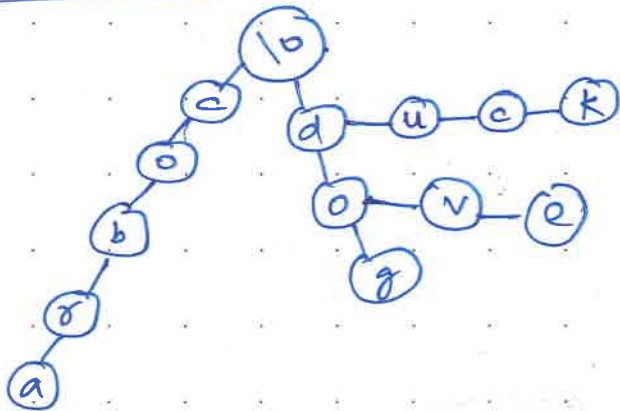
↳ We will output smallest prefix ~~sub~~ substring of each string in words[] which ~~doesn't~~ ~~do~~

is not common in other strings.
Ex:- "d" in "dog" is smallest prefix but

is present in "dove" & "duck" also.
"do" is also not possible as it is also

common in "dove".
So we output "dog" - smallest prefix possible (acc. to condn).

NOTE:- Whenever in a question we get to
• find something related to prefix.
✓ Prefix Tree aka Trie - for the win



• If "cobras" would have existed then "cobra" would have no prefix $\therefore$ output = -1.

Approach

⇒ Count no. of words starting from a given node.

- Ex:- For "cobra", At node "c" it doesn't have ~~any~~ ~~child~~ more than one child. $\therefore$ we directly print "c" as smallest prefix.
- For "dog" at node "d" it has more than 1 children ~~so~~ so we move ahead till we get only 1 child. else print -1.

Max XOR Pair

Find a pair with maximum XOR

Input[] = {3, 10, 5, 25, 2, 8}

Output = $5 \oplus 25 = 28$

Brute force $\rightarrow O(N^2)$ [Time], $O(1)$ [Space]

Better approach

We maintain variable ans. (store max XOR pair value)

(1) ans = 3

(2) When 10 comes,
ans = $3 \oplus 10$

(3) When 5 comes,
it checks previous values and find pair which forms max XOR pair.
ans = $5 \oplus 10$

and so on...

3	→	00011
10	→	01010
5	→	00101
25	→	11001
2	→	00010
8	→	01000

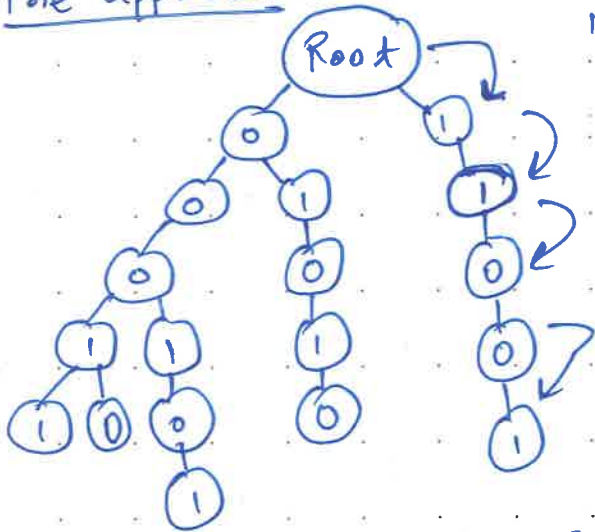
Fig 1.1

But for each • next element if we check the whole array till now, for the max. no. pair. • T.C. will be $O(n^2)$.

So we can create 2D array of each row containing binary of each elements, like in Fig 1.1.

like in Fig 1.1.
~~But~~ and then either we can find prefixes
of each no. or carry out some diff.
opⁿ. But it will take space $O(n^2)$

~~we can~~
Trie approach (saves space & efficient) $[O(N)]$



1) ~~max~~ Build
Trie of all
binary of elements
we have.

2) Now for ~~every~~ a j^{th} element if we want to check other previous elements we have to sort them.

For ex :- $\begin{cases} 3 \\ 10 \\ 5 \\ 25 \\ 2 \end{cases}$ we traversed and performed same opⁿ for this much elements.

and max so far = $5 \cdot 25 = 28$
we get:

Now, we will check for 8.

whether there exist a pair ~~which~~ of
8 with any element in $\{3, 10, 5, 25, 2\}$
such that MAX XOR pair ~~can~~ can
be found.

$\therefore 8 \rightarrow 01000$

→ 01 00 0
So we want to maximize the
XOR of the element with 8.

In our tree, we look opp. bit of 1st bit ~~of~~ of 8 i.e. "0" so we look up for 1.

General Rule:-

General Rule:- we traverse through bits of 8.

01000 →

2) look up for negation ($\sim$) of bit at current posⁿ, in the trie.

3) If its present in hash map of current node in trie then select that node. else, continue with the only node we have. (we have no choice)

4) The no. we got will be the max XOR pair with 8. $\therefore$ max so far

✓. 5) we will keep updating max_so_far
xor pair also

Min XOR Pair

{ 5, 15, 7, 3, 18 }

(1) Sort $\Rightarrow n \log n$

{ 3, 5, 7, 15, 18 }

(2) Take immediate pairs. It will give minimal XOR pairs.

(3) $\min \text{ XOR} = \min (3^5, 5^7, 7^{15}, 15^{18})$

T.C. $\Rightarrow O(n \log n + n) = O(n \log n)$.

(Continuation) MAX XOR Pair.

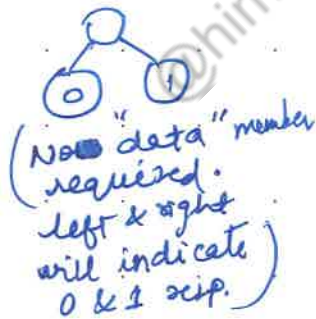
Implementation

```
class node {
public:
    node *left; // for 0
    node *right; // for 1
};
```

```
class trie {
    node *root;
```

```
public:
    trie() {
        root = new node();
    }
```

// insert function
// 5 = 00000101 [32 bit integer]



we are taking max len of binary = 32 (insertion from MSB)

```
void insert(int n) {
    node *temp = root;
    for (int i = 31; i >= 0; i--) {
        int bit = (n >> i) & 1;
        if (bit == 0) {
            if (temp->left == NULL) {
                temp->left = new node();
            }
            temp = temp->left;
        } else { // bit is 1
            if (temp->right == NULL) {
                temp->right = new node();
            }
            temp = temp->right;
        }
    }
}
```

extracts bits at ith posⁿ from LSB.

loop to insert 0 or 1 of 32 bit integer. (from MSB)

So that we can select max

// func which helps to find max xor

```
int max_xor_helper(int value) {
    int current_ans = 0;
    for (int j = 31; j >= 0; j--) {
        int bit = (value >> j) & 1;
        if (bit == 0) {
            // find a complementary value
            if (temp->right != NULL) {
                temp = temp->right;
            }
            current_ans += (1 << j);
        } else {
            temp = temp->left;
        }
    }
    // if current bit is 1.
    if (temp->left != NULL) {
        temp = temp->left;
        current_ans += (1 << j);
    }
}
```

For value = 8
01000
The func will look for '1' (temp->right) to maximize its XOR.
So the contribution of set bit at jth posⁿ = 2^j (1 < j)

Case where complement doesn't exist, so we go on what we have.

"Health is a relationship between you and your body."

```

else {
    temp = temp → right;
}
}
return current - ans;
}

```

```

int max-xor(int &input, int n) {
    int max-xor = 0;
    for (int i = 0; i < n; i++) {
        int val = input[i];
        insert(val);
        int current-xor = max-xor-helper(val);
        cout << current-xor << " "; // (Test)
        max-xor = max(current-xor, max-xor);
    }
    return max-xor;
}

int main() {
    int input[] = {3, 10, 5, 25, 2, 8};
    return 0;
}

```

O/P:- 0 9 15 28 27 17 | 28

↓ ↓ ↓ ↓ ↓ ↓

3^3 3^10 5^10 5^25 2^25

Max-xor pair 5^25

✓ Traversing through array of elements = $O(N)$

Traversing through Trie (always height 32) ⇒ $O(\text{constant})$ ⇒ $O(1)$.

$$\therefore T.C. = O(N) * O(1)$$

$$T.C. = O(N)$$

Hashing Questions

① Find if array contains a subarray whose sum is 0 or not.

for ex:-

6	-1	2	-1	2	-1
---	----	---	----	---	----

Prefix sum array

≡

6	5	7	6	8	7
---	---	---	---	---	---

~~we can use unordered map~~

Repeating elements can tell us whether subarray with sum 0 exists or not

≡

2	-1	-1
---	----	----

↓

2	1	0
---	---	---

→ for ex:- if prefix sum array contains zero at any pos then there exist subarray with zero sum again.

• We can use unordered-map() to store T.F. ~~repeated~~ ~~of element~~ ~~which occurs the very first~~ ~~for the~~ ^{already} visited element else F.

• We can also use unordered-set.

```

for (0 → n) {
    pre += arr[i];
    if (pre == 0 or s.find(pre) != s.end()) {
        return true;
    }
    s.insert(pre);
}

```


2) length of longest subarray with sum 0

0	1	2	3	4	5
0	-1	1	2	5	-5

cumulative sum →

0	-1	0	2	7	2
---	----	---	---	---	---

- Subarray with zero sum
- upto 0 ($c_sum[i] = 0$)
 - between (x, x]

(1) At $i=0$ & $i=2$

csum becomes 0

∴ sub array size in both cases

For $i=0$, len of subarray = $i+1 = 1$

For $i=2$, len of subarray = $i+1 = 3$

(2) For $j=5$

$$\text{len} = j - i$$

index where

$$c_sum = 2$$

$$= 5 - 3 = 2$$

* always store index of occurrence
first occurrence

↓
because we want longest subarray

Algo

1) we will maintain max-len & cur-len variables.

2) Build prefix sum array.

	c sum	idx	cur-len	max-len
1	0	→ 0	0+1	1
3	-1	→ 1		
7	0	→ 2	2+1	3
2	2	→ 3		
	7	→ 4		
	2	→ 5	(5-3)	3

$$[0 \rightarrow 6 \quad (6+1)]$$

$$(6+1)$$

$$7$$

$$\therefore o/p := 7$$

1/2/3/4/5/6/7

1/2/3/4/5/6/7

1/2/3/4/5/6/7

3) longest subarray with sum k

0	1	2	3	4
1	-1	5	-2	3

(k=3)

0	1	2	3	4
1	0	5	3	6

Case (1) if $c_sum[i] = k$
then $cur_len = i+1$

Case (2)

For ex: $a_1, a_2, a_3, \dots, a_n$

$c_sum \rightarrow c_1, c_2, c_3, \dots, c_i, c_{i+1}, \dots, c_j, \dots, c_n$

$$c_j - c_i = K$$

→ subarray with sum K

$$c_i = c_j - K$$

∴ For every j, we will check if

$(c_sum[j] - K)$ exist in the

array traversed so far

if yes then $(j-i)$ = len of subarray with sum K

(index where $c_sum[i] - K$ is there)

For given array "a[]",

sum idx look for (sum)

1 → 0 -2 X

0 → 1 -3 X

5 → 2 2 X

3 → 3 → case (1)

cur-len = 3+1 = 4

6 → 4

3 ✓ case (2)

cur-len = 4-3 = 1

O/P: - max-len = 4

Code

```
for (0 → n) {
    unordered_map<int, int> m;
    pre = 0;
    len = 0;
    if (pre == k) {
        len = max(len, i+1);
    }
    if (m.find(pre - k) != m.end()) {
        len = max(len, i - m[pre]);
    }
    else {
        m[pre] = i; // store first occ
    }
    return len;
}
```

(The above implementation works for all k)

⇒ longest consecutive subsequence

① [10, 4, 20, 1, 3, 3, 2, 19]

[20, 19] → subsequence with adjacent element being consecutive.

10 → being single element subseq.
(also follows the condn.)

4, 1, 3, 2 → 1, 2, 3, 4

(also follows the condn.)

Approach ① [Sorting]

(1) Sort the arr

[1, 2, 3, 3, 4, 10, 19, 20]
4 1 2
max 4

(N log N)

(In above approach we gradually increase window size by checking if the current element equals to / one more than previous element) [Cur-len & max-len variables are used]

(2) for (i = min element in array → max element in array) {
[1, 2, 3, 4, 5, 6, ..., 20] → unordered-set / unordered-map
if (m.get(i) > 0) {
cur-len ++; }
else {
ans = max(ans, cur-len);
cur-len = 0; }
}

T.C. ⇒ if (max - min) > n

Then O(n) >>> ✓

Approach ② [Hashing] ~~Unordered-map~~ (Unordered-map)

- Tricky (So may hard to think) (O(N))
~~14, 5, 1, 2~~

When you insert a new element

① left expansion $\xrightarrow{x+1}$ (len1+1)

② New streak of len1 $\xrightarrow{x}$ (1)

if we can't able to find

$x-1$ or $x+1$ element

in pre-existing streaks.

Then we will make a fresh streak.

③ Right Expansion $\xrightarrow{x}$ (len2+1)

- If $(x-1)$ element already exist in a streak then expand that streak rightwards.

④ Join left & right

$\dots [x-1] \quad [x] \quad [x+1] \dots$

if both $(x-1)$ & $(x+1)$ already exist in two separate streaks so we will insert x in b/w and now it forms a complete streak of bigger length. $(len1 + len2 + 1)$

* we only maintain current length of whole streak in leftmost & rightmost element of streak

14, 5, 1, 2, 6, 7, 8, 9, 3, 4

Hash map
 element-len

14 - 1

5 - 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5

1 - 1 $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 9

2 - 2

6 - 2

7 - 3

8 - 4

9 - 5 $\rightarrow$ 9

3 $\rightarrow$ 3

4 $\rightarrow$ 9

1	2	3	4	5	6	7	8	9
---	---	---	---	---	---	---	---	---

14

2	3	9	2	2	3	4	5
---	---	---	---	---	---	---	---

3	4	5	9
---	---	---	---

① when 2 comes up $(2-1) = 1$ already exists in a streak of len 1 so we update

$m[2-1]++;$

and $m[2] = m[2-1] + 1;$

② when 7 comes up, we have $(7-1) = 6$ present in a streak so ~~if~~ ~~mean~~

$[7 \rightarrow 3] \cdot m[7-1] = 2$

so though this we will $m[7-2]++;$ (Head updated)

we will update head & tail of streak only because we don't require mid elements to calculate the length of streak.

③ when 4 comes up we have

3 & 5 both elements
 so now streak length $= m[4-1] + m[4+1] + 1$

"A good laugh and a long sleep are the best cures in the doctor's book."

Implementation (DIY)

Use unordered-map to store the length of particular element.

After the loop, ~~for each element~~

```
int maxx = 0;
for (auto p : m) {
    maxx = max(maxx, p.second);
}
```

Approach ③ (Unordered-set)

- Easier Sol'n

$O(N)$ ✓

a [14, 5, 1, 2, 6, 3, 7, 8, 9, 13, 15, 11, 12, 13]
X ✓ ✓ X X X X X X X ✓ X X

Algo

(1) For ~~a[i]~~ store all the elements in unordered-set S.

(2) Now, for $a[i]$, if $a[i]-1$ is there in the unordered-set then skip this element.

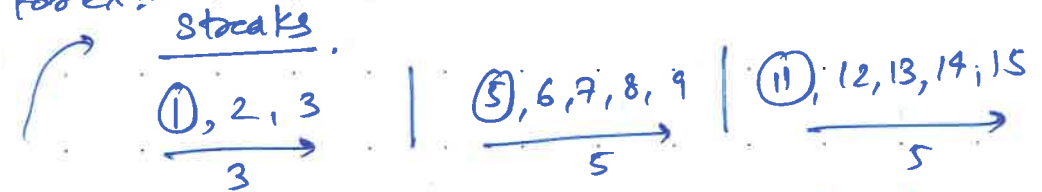
(3) If $a[i]-1$ doesn't exist then it is likely that $a[i]$ is the head of one of the possible streaks.

Then we check till which no. it has streak of. we increase $a[i]$ by 1 and check if it exists in set or not.

(4) By performing ~~same~~ step 2 on words repeatedly on each element.

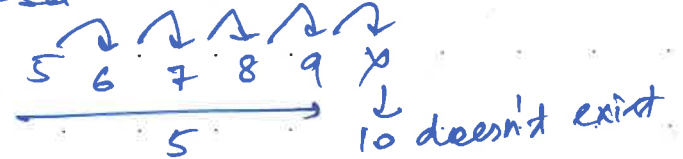
(5) At last we will get max-len of streak.

For ex:-



- When we check for 14, 13 already exists so we skip
- For 5, ~~we check~~ 4 doesn't exist so we check successors it has in adjacent

unordered-set



- For 1, we again perform same step like we did for 5.

Code

```
int findLongestSubseq(int a[], int n) {
    unordered_set<int> s;
    for (int i = 0; i < n; i++) {
        s.insert(a[i]);
    }
    int max_streak = 1;
    for (int i = 0; i < n; i++) {
        if (s.find(a[i]-1) == s.end()) {
            int next_no = a[i] + 1;
            int streak_len = 1;
            while (s.find(next_no) != s.end()) {
                next_no += 1;
                streak_len += 1;
            }
        }
    }
}
```



```

max-streak = max(streak-len, max-streak);
}
return max-streak;
}

```

Min. Window Substring Concept

Q. find the smallest window in a string containing all characters of another string.

string → "helloeeo world"
pattern → "eeeo"

(1) Brute Force

1) First find all the substrings of the string. $O(N^3)$ / $O(N^2)$

2) Then run another loop through the pattern to check if it holds the "brd". $O(N)$

Total - $O(N^3)$ / $O(N^4)$.

All substrings of string
Three loops = $O(N^3)$
method
Optimised = $O(N^2)$
(uses previous substring) $\downarrow$
(Best possible)

2) Better Approach

We will use sliding window
1) Handle corner case in the beginning when pattern > string

2) Make a pattern freq array & init it with freq. of characters.

3) Expand toward right to get the window, once you get the window you contract from the left to avoid un-required character (either the char is not present in the pattern or its frequency in the string is higher than what is required)

4) Every time you get a window, update your min window by doing a comparison b/w current window and min window you got till now.

5) Output the min window length.

Code

```

string find-window(string s, string pat) {
    int sl = s.length();
    int pl = pat.length();
    if (pl > sl) return "None";
    int min-len = INT_MAX;
    int start-len = -1;
    int fs[256] = {0}; // freq. maps
    int fp[256] = {0}; // (These are 256 ASCII characters)
    for (int i = 0; i < pl; i++) {
        char ch = pat[i];
        fp[ch]++;
    }
    int cnt = 0;
    int start = 0;
    int minlen = INT_MAX;
    for (int i = 0; i < sl; i++) {
        char ch = s[i];
        fs[ch]++;
        // Check cnt of characters matched
        if (fp[ch] != 0 and fs[ch] <= fp[ch]) {
            cnt++;
        }
        // Sliding window (Exp & Contr. + Update the min len window)
        while (cnt == pl) {
            // if we already have enough char we don't inc. count
            if (i - start + 1 < minlen) {
                minlen = i - start + 1;
                start-len = start;
            }
            char c = s[start];
            fs[c]--;
            if (fp[c] != 0 and fs[c] < fp[c]) {
                cnt--;
            }
            start++;
        }
    }
    if (start-len == -1) return "None";
    return s.substr(start-len, minlen);
}

```

// if all the char. match

if (cnt == PL) {

// start shrinking the window

// loop to remove all un-wanted characters

char temp = s[start];

while (FP[temp] == 0 or FS[temp] > FP[temp]) {

FS[temp]--;

start++;

temp = s[start];

}

// window size

int window_len = i - start + 1;

if (window_len < min_len) {

min_len = window_len;

start_idx = start;

}

if (start_idx == -1) {

return "None";

}

}

(Map FS will store the count of each char. present in current window)

Pattern length

if the current character count is greater than pattern's count, so we can remove it from sliding window.

Point till we removed elements from window

You can also print the string (shortest string)

string ans = current ans; and other modification code

string ans = s.substr(start_idx, min_len);
return ans;

Subarray with Max XOR [Trie]

a = {3, 10, 4, 25, 2, 8}

① Brute Force $O(n^3)$

② Cumulative XOR, $O(n^2)$

make a cumulative XOR array

C-XOR

XOR(L, R) = C-XOR(R) ^ C-XOR(L-1)

Two loops For ex:-

$(3 \oplus 10 \oplus 4 \oplus 25) \oplus (3)$
 $= 10 \oplus 4 \oplus 25$

Find pair such that their XOR(L, R) is max.

③ Using max XOR pair (Trie) $O(n^2)$

3 → a

3 ^ 10 → b

3 ^ 10 ^ 4 → c

3 ^ 10 ^ 4 ^ 25 → d

3 ^ 10 ^ 4 ^ 25 ^ 2 → e

3 ^ 10 ^ 4 ^ 25 ^ 2 ^ 8 → f

cumulative XOR array

cum-xor	a	b	c	d	e	f	g
---------	---	---	---	---	---	---	---

- Find MAX-XOR pair of this cum-XOR array.

For ex:- if b & e forms Max XOR pair that means $b \oplus e = \text{max XOR possible}$

$$\therefore b \oplus e = (8 \oplus 16) \oplus (8 \oplus 16 \oplus 4 \oplus 25 \oplus 2) = 4 \oplus 25 \oplus 2$$

4	25	2
---	----	---

MAX XOR subarray

T.C. $O(N) * O(N)$

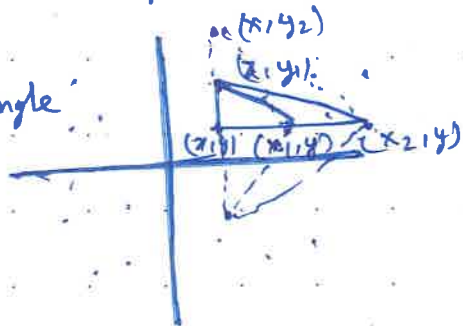
↓
Cum.
XOR
array
build

→ Max XOR pair using trie.

Right Angled Triangles [Hashing]

Given N points in a 2D Cartesian Plane, How many right angled triangles are there such that perpendicular or base is parallel to X or Y axis?

→ right angle



We will take (x_1, y_1) as pivot to draw triangles.

n_1 points
↓
(vertical level)

$x, y \rightarrow$ We take this as pivot (right angle pt.)

n_2 points → (horizontal level)

then just we have to choose one point in horizontal (out of $n_1 - 1$ points) & one pt. from vertical (out of n_2 pts.)

$$n_1 - 1 C_1 \times n_2 - 1 C_1 = (n_1 - 1)(n_2 - 1)$$

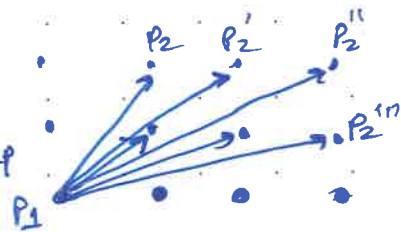
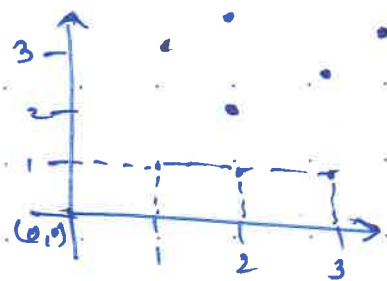
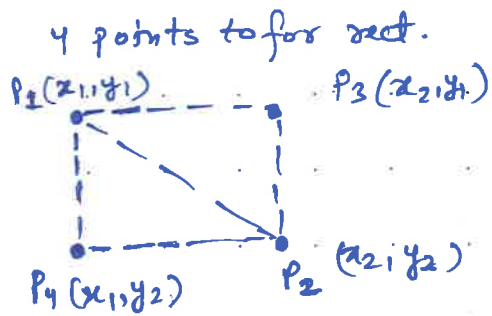
So we will maintain hashmap to store for specific x how many point lies $(x, y_1), (x, y_2) \dots (x, y_n)$ and same for specific y $(x_1, y), (x_2, y) \dots (x_n, y)$

Ex:- For $x = 2$, there are 3 points $\therefore h_1[2] = 3$
For $y = 1$, there are 4 pts $\therefore h_2[1] = 4$

Count Rectangles [Hashing]

Given N Cartesian Points, find the no. of axis parallel rectangles that can be formed by them.

points = $[(0,0), (0,1), (1,1), (1,0), (2,1), (2,0), (3,1), (3,0)]$



Possible P_2 (opp. diagonal pt.) which will form rectangle. & then P_3 & P_4 can easily be found using P_1 & P_2 .

Redundant $(P_1, P_2) \leftrightarrow (P_2, P_1)$ can form a rect. so accordingly we will deal.

```
class Point {
public:
    int x, y;
    Point() {}
```

```
    Point(int x, int y) {
        this->x = x;
        this->y = y;
    }
```

class for coordinates

Compare class for set. as reference

```
int countRects class Compare {
```

```
public:
    bool operator()(Point p1, Point p2) {
        if (p1.x == p2.x) {
            return p1.y < p2.y;
        }
```

```
return p1.x < p2.x; }
```

```
int countRects(vector<Point> coords) {
```

```
    int n = coords.size();
```

```
    set<Point, Compare> S;
```

```
    int ans = 0;
```

```
    for(auto it = S.begin(); it != S.end(); it++)
```

```
        for(auto p: coords) {
```

```
            S.insert(p); } // Insert all the coordinates
```

```
for(auto it = S.begin(); it != S.end(); it++) {
```

```
    for(auto jt = next(it); jt != S.end(); jt++) {
```

```
        Point p1 = *it;
```

```
        Point p2 = *jt;
```

```
        if (p2.x == p1.x || p2.y == p1.y) {
```

```
            continue; }
```

```
        // if p1 comes out to be equal to p2 then skip
```

```
        Point p3(p2.x, p1.y);
```

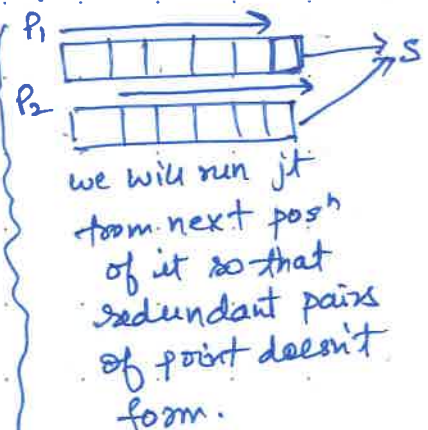
```
        Point p4(p1.x, p2.y);
```

```
        if (S.find(p3) != S.end() &&
```

```
            S.find(p4) != S.end()) {
```

```
            ans++; }
```

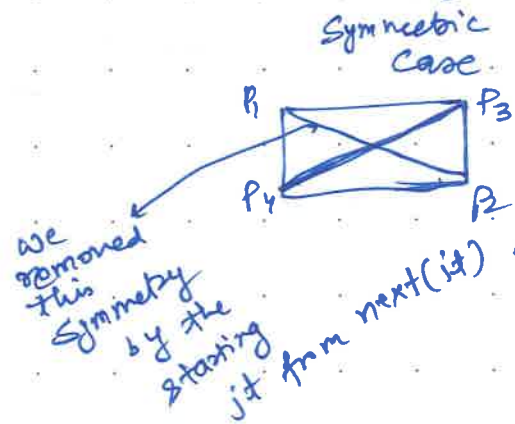
when comparison done inside STL then ~~the object~~ the class with $()$ overload is called.



Manipulation of P_1 & P_2 to get P_3 & P_4 .

// if P_3 & P_4 exists inc. ans by 1

return ans/2; $\rightarrow$ We removed redundant pairs of P_1 & P_2 but we can get redundant pairs of P_3 & P_4 . Thus, reduce by half.



but if in the same double loop if we get P_3 & P_4 as points then we will find P_1 & P_2 by manipulation of P_3 & P_4 . So same rect, we will get.

Code

```
class Graph {
    int v;
    list<pair<int, int>>> l;
public:
    Graph(int v) {
        v = v;
        l = new list<pair<int, int>>>[v];
    }
    void addEdge(int u, int v, int cost) {
        l[u].push_back(v);
        l[v].push_back(u);
    }
}
```

Fast Exponentiation using bitmasking

code

```
int n, a;
cin >> n >> a;
while(n > 0) {
```

We are finding 2^n in $O(\log(n))$.

```
if(n & 1) {
```

```
    maxNo = maxNo * a;
    maxNo %= MOD;
    maxNo = (maxNo + MOD) % MOD;
```

(Try XOR EQUAL for understanding)

```
}
a = a * a;
a %= MOD;
n >>= 1;
```

```
}
cout << maxNo % MOD << endl;
```

Thumb rule

$$(x + \text{MOD}) \% \text{MOD} = x \% \text{MOD}$$

For instance, if we get $\text{maxNo} < 0$ then we add MOD to make it positive.

In case $x \% \text{MOD}$ comes up negative so we do $(x + \text{MOD}) \% \text{MOD}$ to make it positive.

$$0 < (x + \text{MOD}) \% \text{MOD} \leq \text{MOD} - 1$$

-ve

Find all factors of a number n

#define MAX 5'00'000

int lowest_prime_factor[MAX];

void init()

Efficient approach given ahead.

// This func stores lowest prime factor for the no.

For ex:-

- for multiples of 2, we store 2.
- when 27 comes up we get 3 from this array and so on like this

```
for (int i=2; i<MAX; i++){
    if (i%2){
        lowest_prime_factor[i] = i;
    } else {
        lowest_prime_factor[i] = 2;
    }
}

for (int i=3; i<MAX; i+=2){
    if (lowest_prime_factor[i] == i){
        for (int j=i+1; j<MAX; j+=2*i){
            if (lowest_prime_factor[j] == j){
                lowest_prime_factor[j] = i;
            }
        }
    }
}
```

```
void helper (vector<pair<int, int>> &vec, int i, int val, vector<int> &ans){
    if (i == vec.size()) {
        ans.push_back(val);
        return;
    }
```

0 → N knapsack

```
for (int cnt=0; cnt<=vec[i].second; ++cnt){
    helper (vec, i+1, val, ans);
    val *= vec[i].first;
}

// We move to next prime factor
// afterwards update val.
```

// The above function gives all factors for the no.

Ex:- for vec = {{2,1},{5,1}}

ans = [1, 2, 5, 10]

occurrences
prime factor

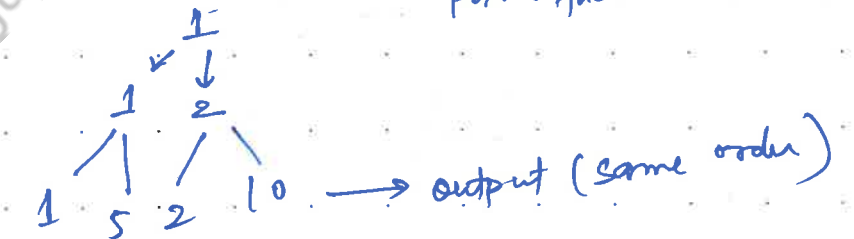
```
helper (vec, i+1, val, ans);
for (int cnt=1; cnt<=vec[i].second; ++cnt){
    helper val *= vec[i].first;
    helper (vec, i+1, val, ans);
}
```

Case where we exclude (only consider 1)

Consider the prime factor

We can do this way also.

All next prime factors



```
vector<int> factors (int n){
    vector<pair<int, int>> vec;
    while (n != 1){
        int cnt = 0;
        val = lowest_prime_factor[n];
        while (lowest_prime_factor[n] == val){
            n /= val;
            ++cnt;
        }
        vec.push_back({val, cnt});
    }
    vector<int> ans;
}
```

gives prime factors with count

"Your body hears everything your mind says."


```

helper (vec, 0, 1, ans);
return ans; }

```

I/P:-
10

O/P:-
{1, 5, 2, 10}

Explain ()

~~Knapack 0/1~~ → We only have to either include or exclude
↓
↪ recursive calls.

• ~~Knapack 0/1~~ We have plethora of choices to choose from so we run loop from 0 to N and consider all case where first we exclude and afterwards include 1 item, 2 item ... N item.
↓
(N+1) recursive calls

Take input in Level order (Tree)

node* buildLevelorder () {

queue<node*> q;

int x;

cin >> x;

if (x == -1) { return NULL; }

node* root = new node(x);

q.push(root);

while (!q.empty()) {

node* temp = q.front();

q.pop();

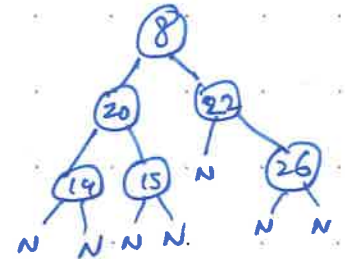
cin >> x;

if (x != -1) { temp->left = new node(x);
q.push(temp->left); }

cin >> x

if (x != -1) { temp->right = new node(x);
q.push(temp->right); }

return root; }

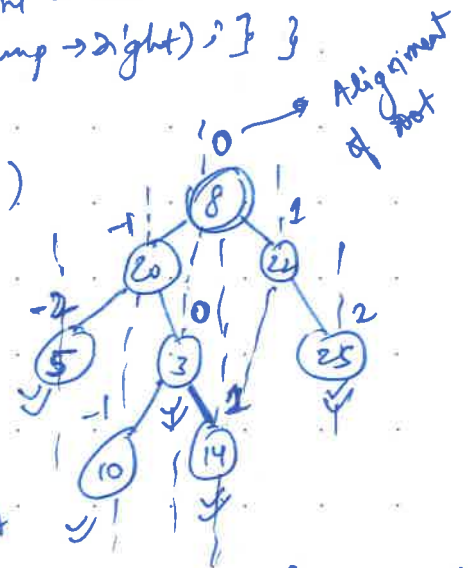


I/P:-

8 20 22 19 15 -1 26
-1 -1 -1 -1 -1 -1

Bottom view (Code on CB)

We use ~~unordered map~~
map<int, pair<int, int>>
↓ ↓ ↓
alignment data level
pos



In each vertical alignment we print the ~~data~~ node which is present at highest level (bottom-most)

O/P:- 5 10 3 14 25

- Getting all distances from each node to another in Graph. (Using BFS)

* we can call BFS SSSP for each node as source and store the distances in a matrix $dis[u][v]$ (where $u = \text{current src}$
 $v = \text{current destination}$)

Code

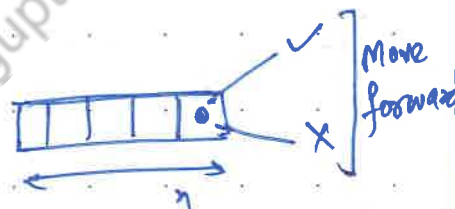
```
vector<vector<ll>>> dis;
void bfs(ll src, vector<ll> &l,
vector<ll> &dis vector<ll> &dis) {
    since the nodes are 1 to N so we can directly use vector & use hashing
    vector<ll> count(n, INT_MAX);
    queue<ll> q;
    dis[src][src] = 0;
    q.push(src);
    count[src] = 0;
    while (!q.empty()) {
        ll node = q.front();
        q.pop();
        for (auto nbr : l[node]) {
            if (count[nbr] == INT_MAX) {
                q.push(nbr);
                count[nbr] = count[node] + 1; // update no. of edge on nbr
                dis[src][nbr] = count[nbr];
            }
        }
    }
}
```

```
void solve() {
    take input();
    vector<ll> l(n+1, vector<ll>(n+1));
}
```

```
for (ll i = 1; i <= n; i++) {
    bfs(i, l, dis);
}
```

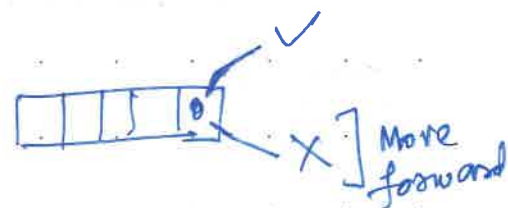
⇒ Unbounded Knapsack (0-N Knapsack)
(Repetition of items allowed)
Given a knapsack of capacity c and a set of n items with certain value $val[i]$, weight $wt[i]$. Here we are allowed to use unlimited no. of instances of an item

0-1 Knapsack



In 0-1, when we either include or exclude. we move to next item i.e. arr of size $n-1$.

0-N Knapsack



In 0-N, we only move to next of array (next item) when we exclude the current item. but in case of include we have the option that we can again take that same item again & again.

Top-down approach

```
ll knapsack (ll *val, ll *wt, ll n, ll c,
            vector<vector<ll>> &dp) {
```

```
    if (n == 0 || c == 0) {
        return dp[n][c] = 0; // if capacity = 0
    } // or no item is considered yet then
```

```
    if (dp[n][c] != 0) {
        return dp[n][c];
    }
```

// lookup

```
    ll inc = 0, exc = 0;
```

```
    if (size[n-1] <= c) {
```

```
        inc = val[n-1] + knapsack(val, size, n, c - size[n-1], dp);
    }
```

// excluded - we move to the rest of the array and we don't need to consider the current item.

```
    exc = knapsack(val, size, n-1, c, dp);
```

```
    return dp[n][c] = max(inc, exc);
```

```
}
```

Bottom-up

```
void solve () {
```

// inputs

```
    vector<vector<ll>> dp(n+1, vector<ll>(c+1, 0));
```

```
    cout << knapsack(val, size, n, c, dp);
}
```

Bottom-up approach

```
ll knapsack (ll *val, ll *wt, ll n, ll c, &dp) {
```

```
    for (ll i = 0; i < n; i++) {
```

```
        for (ll j = 0; j <= c; j++) {
```

```
            if (wt[i-1] <= j) {
```

```
                dp[i][j] =
                    max(dp[i-1][j],
                       dp[i-1][j - wt[i-1]] + val[i-1]);
```

```
            }
```

```
        else {
```

```
            dp[i][j] = dp[i-1][j];
        }
```

```
    }
    return dp[n][c];
}
```

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0					
2	0					
3	0					
4	0					

(either we can initialize it earlier or with a cond'n)

only change once
0-1

(we don't go back to previous item, but remain at this item (row) only)

Find all factors of a number n [Efficient Approach]

#define MAX 1'00'005

vector<vector<int>> factors(MAX);

void init()

{ for (int i=1; i<MAX; i++) {

for (int j=1; j<MAX; j+=i) {

factors[j].push_back(i);

}} }

Time complexity

$$\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \dots + \frac{n}{n} < \frac{n}{1} + \frac{n}{2} + \frac{n}{2} + \frac{n}{4} + \frac{n}{4} + \dots$$

$$\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \dots + \frac{n}{n} < \frac{n}{1} + 2\left(\frac{n}{2}\right) + 4\left(\frac{n}{4}\right) + 8\left(\frac{n}{8}\right) + \dots$$

$$\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \dots + \frac{n}{n} < n + n + n + n + \dots + n$$

$$\boxed{\frac{n}{1} + \frac{n}{2} + \frac{n}{3} + \frac{n}{4} + \dots + \frac{n}{n} = O(n \log_2 n)}$$

Why is Fibonacci Big $O(2^n)$ instead of $O(\log n)$?

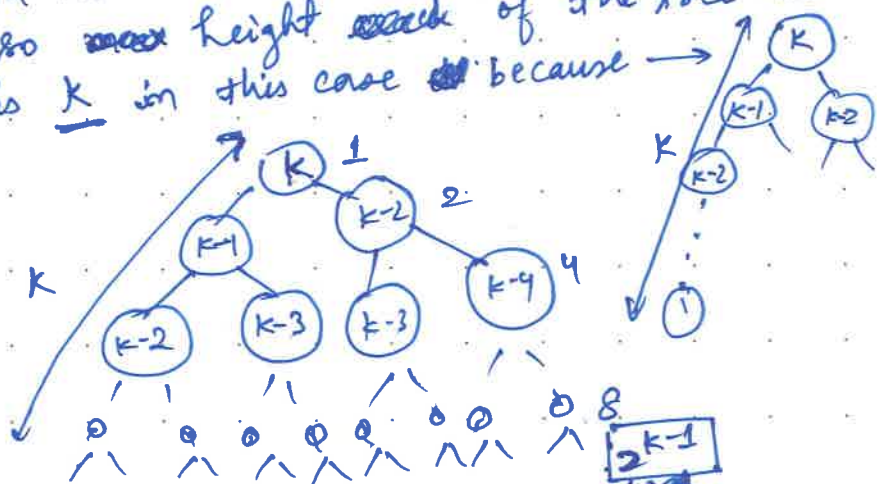
Basic facts about binary tree :-

- No. of leaves n in a binary tree is equal to the no. of non-leaf nodes plus one.

The size of binary tree is thus $2n-1$.

- In case of divide & conquer, the width of tree remains same as input array that is n . Thus height is $O(\log n)$.

- In Fibonacci, we call $f(k-1)$ & $f(k-2)$ so ~~max~~ height ~~max~~ of the tree is k in this case because



Total no. of leaf nodes = 2^{k-1}
in case if we consider perfect binary tree

“Eat to live, not live to eat.”

Q.C. a ~~height~~ ~~width~~

$$T(n) = T(n-1) + T(n-2) + O(1)$$

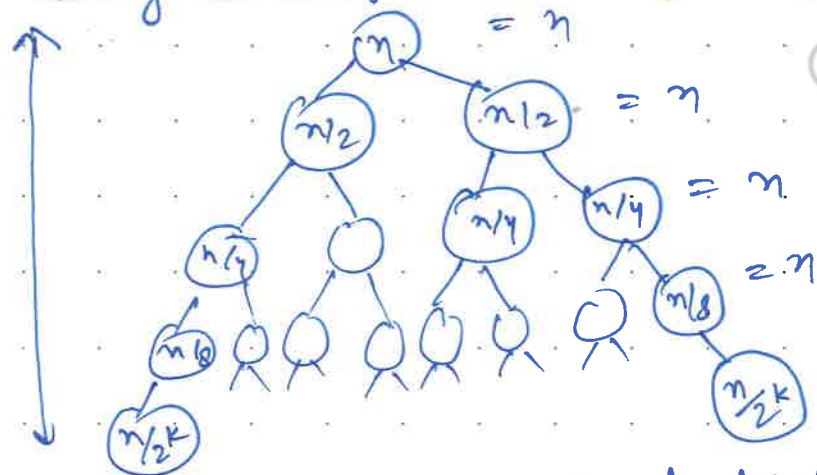
$k(1+2+4+8+\dots+2^{k-1})$
 $k \frac{(2^k - 1)}{2 - 1} = O(2^k)$
 Constant opⁿ?

Ez way of T.C.
 Height of Tree = k
 At each step, no. of nodes gets squared
 $\therefore T.C. \Rightarrow O(2^k)$

Conclusion:-

- In DkC, the input array size is tree's width.
- In Fibonacci, the input array size is height of the tree.

Effective size remains same at each level. k since we are dividing the array in half at each stage. $\therefore O(\log_2 n)$



Since size gets halved at each level.

$$\frac{n}{2^k} = 1 \Rightarrow \log_2 n = k$$

$$T(n) = T(n/2) + T(n/2) + \frac{n}{2}$$

That means at each level $O(n)$ is used, just like merge sort.

$$\therefore T(n) = n \times k$$

$$T(n) = O(n \cdot \log n)$$

[Refer DAA notes]

For array of strings.

geeksforgeeks | geeks | geek | geezer

Sorting T.C.

$$O(m * n \log n)$$

max len of str

no. of strings

Normally in case of

4 | 3 | 1 | 2

T.C - $O(n \log n)$
where each comparison takes only $O(1)$

$$\therefore O(1 * n \log n)$$

KMP Algorithm for Pattern Searching

Given a text $[0..n-1]$ and a pattern $pat[0..m-1]$
write a func search (char pat[], char txt[])
that prints all occurrences of pat[] in txt[]. You
may assume that $n \geq m$.

1) I/P:- txt[] = "THIS IS A TEST TEXT"
pat[] = "TEST"

O/P:- Pattern found at index 10.

2) I/P:- "AABAACAADAABAABA"
"AABA"

O/P:- Pattern found at 0, 9, and 12

-Naive algo

```
void search(string pat, string txt){
```

```
    int n = txt.size();  
    int m = pat.size();
```

```
    for (int i = 0; i <= n-m; i++){
```

```
        int j;
```

```
        for (j = 0; j < m; j++){
```

```
            if (txt[i+j] != pat[j]){  
                break; }  
        }
```

```
        if (j == m){  
            cout << i << endl; }  
    }
```

```
}
```

when no. of
char in txt
left are less
than pat
size, we
don't need
to check for it.

Worstcase

$O(m * (n-m+1))$

KMP Method (For detailed explanation refer gfg)

- The basic idea is whenever we detect a mismatch (after some matches), we already know some of the characters in the text of the next window. We take adv. of this info to avoid matching the characters that we know will anyway match.
- To know how many characters to be skipped, we pre-process pattern and prepare an integer array lps[] that tells us the count of chars to be skipped (using prefix & suffix match count)

LPS $\rightarrow$ Longest proper prefix which is also suffix. A proper prefix is prefix with whole string not allowed.
Prefixes of "ABC" are "", "A", "AB" & "ABC". Proper prefixes are "", "A", "AB".
Suffixes $\rightarrow$ "", "C", "BC" & "ABC".

For each sub-pattern $pat[0..i]$ where $i = 0$ to $m-1$, lps[i] stores length of the max matching proper prefix which is also a suffix of the sub-pattern $pat[0..i]$

NOTE:- If we also consider full string as prefix then, everytime suffix will exist which would be the whole string itself.

Lps calculation of a given pat[]

P₁: abedabeabf
0 1 0 0 1 2 0 1 2 0

P₄: a a a a b a a c d
0 1 2 3 0 1 2 0 0

P₂: abc deabf abc
0 0 0 0 1 2 0 1 2 3

P₃: a a b c a d a a b e
0 1 0 0 1 0 1 2 0 0

String matching (Refer gfg for clear explanation)

String: a b a b c a b c a b a b a b d

pattern:

a	b	a	b	d
0	0	1	2	0

 → lps

o/p:-
found 1
occurrence
at index 10

Code

```
void computeLPSarray(char* pat, int M,  
int* lps) {
```

// length of the prev longest prefix suffix

int len = 0;

lps[0] = 0; // always 0 at index 0

// the loop calculates lps[i] for i = 1 to M-1

int i = 1;

while (i < M) {

if (pat[i] == pat[len]) {

len++;

lps[i] = len;

i++;

}

```
else {  
    if (len != 0) { // Imp step  
        len = lps[len-1];  
        // we want increment i here  
    }  
    else {  
        lps[i] = 0;  
        i++;  
    }  
}
```

```
void kmpSearch(char* pat, char* txt) {  
    int m = strlen(pat);  
    int n = strlen(txt);
```

int lps[M];

// preprocess the lps
computeLPSarray(pat, M, lps);

int i = 0; // index for txt[]

int j = 0; // index for pat[]

while (i < N) {

if (pat[j] == txt[i]) { Expand
j++; // slide the window

i++; }

if (j == m) { // found a pattern

cout << i-j << endl; } // from where this pattern starts in txt

j = lps[j-1];

}

// mismatch after j matches

else if (i < N && pat[j] != txt[i]) {

// Do not match lps[0...lps[j-1]]

// char... match anyway

we move backward by one step
Since window will be slid by one index

```

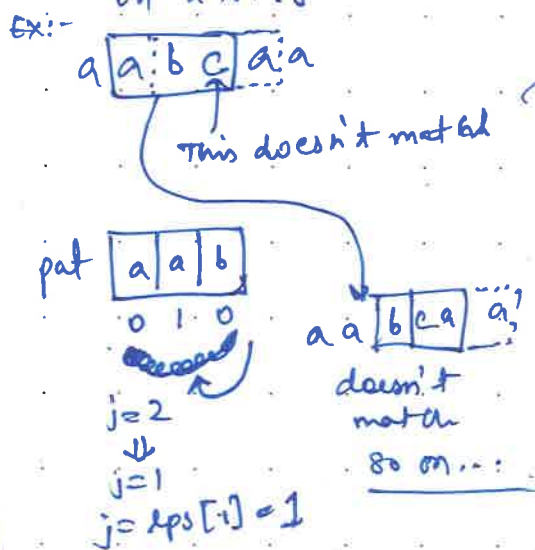
if (j != 0) {
    j = lps[j-1]; } // When we don't find match
else
    i = i + 1;
}
}

```

```

int main() {
    char txt[] = "_";
    char pat[] = "_";
    kmpSearch(pat, txt);
    return 0; }

```



LPS usage * (Example)

⇒ string a = "aabaab"

• proper prefix = "a", "aa", "aaba"

• proper suffix = "a", "aa", "baa", "aaba"

(all substring from right)
(Orientation remains same)

So we get "aa" common.

∴ longest proper prefix which is a suffix = "aa"

Def:- LPS gives me longest LPS till a.size()-1.

LPS at kth posⁿ will tell me longest prefix which is suffix

Alternative method to apply KMP string matching (easy method)

```

pattern[] = "ababd"
text[] = "ababcababababd"

```

- (1) We append any special char to pattern & then text[]
- (2) We will find the LPS of resulting str.
- (3) which will give me the no. of occurrences of pattern[] in text[].

String str = pattern + '\$' + text

str = ababd\$ababcababababd

LPS(str) will return me the

the LPS[] of str:-

a	b	a	b	d	\$	a	b	a	b	c	a	b	c	a	b	a	b	a	b	d
0	0	1	2	0	0	1	2	3	4	0	1	2	0	1	2	3	4	3	4	5

Why we used '\$'?

LPS at kth posⁿ tells us the longest proper prefix which is a suffix also in substring from index 0 to k.

Ex:- abcbabcbab

prefix: abcbab

suffix: bcbab

longest lps = "abcbab"

"Health and intellect are the two blessings of life."

'\$' here works as a partition in LPS[]

str = pattern + '\$' + text

REASON: this puts the value 0 at its index in lps[] which breaks the chance of pattern getting involved in text to get a suffix.

because '\$' is unique and has no '\$' before

- After '\$' it checks for the proper suffix which can be equal pattern which is a proper prefix in this case.
- In str, if we get LPS then it will be the equal to the of the length of pattern that means there is a substring after '\$' which is equal to pattern.

Q why there should be a '\$'?

Ex: - pattern: abcba \$ bcba ← text

LPS ⇒

0	0	0	0	1	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---

abcba \$ bcba

This time we didn't get any bucket after '\$' so no ~~possible~~ occurrence text.

★ But in example (1) we get LPS = abcon. because '\$' was not there and it included pattern also which will give us wrong result.

- If LPS func() finds a LPS ~~which is~~ of length = pattern.size() that means

pattern \$ text

In this region we found a suffix which is equal to prefix ⇒ (pattern)

If this occurs in text suffix substrings then LPS will tell us.

code [Easy method for KMP]

vector<int> LPSfind (string A) {

int n = A.size();
vector<int> lps (n);

lps[0] = 0; // Always 0 at index 0.

int len = 0;

int i = 1;

while (i < n) {

if (A[len] == A[i]) {

len++;

lps[i] = len;

i++;

// If we get another match increment len and store


```

else {
    if (len != 0) {
        len--;
        len = lps[len];
    }
    else {
        lps[i] = 0;
        i++;
    }
}
return lps;

```

// else we move behind to check lps value which will tell us lps till that posn (i.e len-1) and len = lps[len-1] to move len to index where longest prefix ends.

if len == 0 then we move ahead taking lps[i] = 0 would case

```

int KMP (string pattern, string text) {
    int n = pattern.size();
    int m = text.size();
    if (n == 0) return 0;

```

```

    string str = pattern + '$' + text;
    vector<int> lps = LPSfind(str);
    int cnt = 0;
    for (int i = n+1; i < str.size(); i++) {
        if (lps[i] == n) {
            cnt++;
        }
    }
    return cnt;
}

```

we will check text string only

starting index of suffix substring

cnt++ // stores no. of occurrences

i is the index in str but we have to return according to text indexing.

cnt++

return cnt;

```

int main() {
    string pattern = "abcbaba";
    string text = "bcbaba";
    cout << KMP(pattern, text);
}

```

Questions on LPS

- Min. chars to be added at front to make string palindrome.

str = "aacecaaaa"

O/P: - 2

palindrome str = "aaaacecaaaa"

Algo

A = str + \$ + reversed str

A ⇒ aacecaaaa \$ aaaacecaa

proper prefix suffix

- we will check for the LPS substring in above string A to get longest possible proper prefix which is also a suffix in A.

- Reversing the str and then applying LPS in str & revstr allows us to know what is

the max size of ¹subtring in original str. which is a palindrome.

(3) Then $\text{str.size}() - \text{LPS}(A)$ will give us the no. of elements which is extra in original str ^{from the end} so that we can add ~~the~~ same no. of chars at front to get palindrome.

Variant :- Min. chars to be added at back to form palindrome

logic:- we will reverse this original str
and apply some ~~at~~ ~~at~~
and then we just have to find
no. of chars to be added at front.
like before.

★

② Find if a given string can be represented from a substring by iterating the substring "n" times.

fx!

Ex: str = "abababab"

O/P:- true, "ab"

O/P:- true, ab
The give string is 4 times repetition of "ab"

Algo

1) we will find LPS of str.

(2) $\text{str} - \text{LPS}(\text{str})$ will give us the substring which will give us str on str.size repetition.

str - LPs(str).size()

Exp

ab ab ab ab $\rightarrow$ suffix

longest prefix
proper

LPS = 6 ["ababab"]
size() → si

longest prefix proper

size()

size()

string ans = str.substr(0, str.size - LPS);

↓ 8

↓ 6

$\text{ans} = "a.b"$

String matching Algos

1) KMP Algo

2) 2 Algo

3) Rabin-Karp Algo

Tricks (DP)

We find it difficult solve through bottom-up ~~when we~~ but we can easily find out recursive approach which then can be changed into top-down-DP. We can apply bottom-up same as top-down in reverse topological order.

like we do when we apply top-down.

~~the same can be done top down. top down ending started by starting~~

→ In DP Qs where we have to look for all combinations in a string. There are two methods to apply bottom-up DP

1) Length method - here you consider $len=1$, $len=2$, & on till $len=n$ to cover up all substrings length wise using a start point.

2) 2 pointers method - Here, you maintain two pointers i & j and check for all i & j by running double loop.

Q. Minimum Insertion Steps to make a string Palindrome

In one step you can insert any character at any index of the string. Return the minimum no. of steps to make s palindrome.

Input: - "leetcode"
Output: - 0 | Input: - "racecar"
Output: - 5 | Exp: - "racecar"

Code (TOP-DOWN APPROACH)

```
int util(string &s, int i, int j, vector<vector<int>> &dp) {  
    if (i >= j) {  
        return 0; }  
    if (dp[i][j] != -1) { return dp[i][j]; }  
    if (s[i] == s[j]) {  
        return dp[i][j] = util(s, i+1, j-1, dp); }  
    return dp[i][j] = min(util(s, i, j-1, dp),  
                          util(s, i+1, j, dp)) + 1;  
    if (s[i] != s[j]) then we have option of insertion at either posn.  
}  
int minInsertPalin(string s) {  
    vector<vector<int>> dp(s.size(), vector<int>(s.size(), -1));  
    int ans = util(s, 0, s.size()-1, dp);  
    return ans; }
```

WE WILL CHANGE TOP-DOWN TO BOTTOM-UP APPROACH (DP TRICK)

Code (Bottom-up Approach)

```
int minInsertPalin(string s) {  
    vector<vector<int>> dp(s.size(), vector<int>(s.size(), 0));
```

"If you think wellness is expensive, try illness."


```

int n = s.size();
for (int i = 0; i < n; i++) { // for len 1 - always 0
    dp[i][i] = 0;
}
for (int i = n-2; i >= 0; i--) {
    for (int j = i+1; j < n; j++) {
        if (i+1 > j-1) {
            dp[i][j] = 0;
        }
        if (s[i] == s[j]) {
            dp[i][j] = dp[i+1][j-1];
        }
        else {
            dp[i][j] = min(dp[i+1][j], dp[i][j-1]) + 1;
        }
    }
}
return dp[0][n-1];

```

current i will call i+1 for values so we run loop from end i.e n-2 because j > i always.

j will call j-1 for values so we will run loop next posn to i till last.

in case $i == j-1$ and $s[i] == s[j]$ so i will become more than j (violation) so for that case.

if $s[i] != s[j]$
for ex:-
 $s[i] = 'b'$
 $s[j] = 'd'$
we can insert 'd' before 'b' or we can insert 'b' after 'd'. we will call both ways to find which gives min sol. (+ 1) → for current insertion.

INTERNATIONAL TRAVEL INSURANCE



Hospitalization Cover -
IPD & OPD



Pre-Existing Ailments
covered in case of Life
Threatening Conditions



Double Sum Insured -
Accidental Hospitalization



Coverage for Loss of
Passport & Checked-in
Baggage



Trip Delay &
Cancellation



Upgradation to
Business Class

Conditions Apply

explore

INTERNATIONAL TRAVEL & HEALTH INSURANCE FOR STUDENTS



In-Patient Hospitalization



Coverage at
Home Country



Dental Expenses



Loss of Passport
& Laptop



Loss/Delay of Checked-in
Baggage



Sponsor Protection

Conditions Apply

method-2) If we take LCS of current string & its reversed then it will give me longest palindrome subsequence and original string.size() - LCS(str, rev) will give us min insertions.

• Counting

• Occurrence of an element at i^{th} index in all contiguous subarrays of an array.

Ex:- In $[1, 2, 3, 4]$

2 occurs in $[2], [1, 2, 3, 4], [1, 2], [2, 3], [1, 2, 3], [2, 3, 4]$

Formula = $(i+1) * (n-i)$

no. of
partitions
to right of i

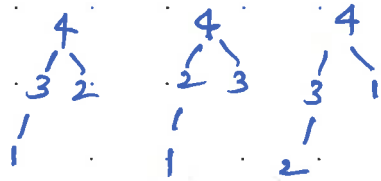
no. of partitions (barriers)
to left of i

EX:-

Q. find XOR of all contiguous subarrays in an array (Interviewbit) $\rightarrow$ [XOR-ing the subarrays]

No. of ways to form a heap with n distinct integers

Ex: $n=4$ o/p:- 3



Logic

Root is fixed i.e. max. no. = n .

Now $(n-1)$ elements left and main observation here is that the structure of the heap nodes will remain same for all instances, but only the values of in the nodes will change. because the max heap properties $\rightarrow$ (Tree is Complete binary tree)

let l = elements in left subtree
 r = elements in right subtree

also $n-1 = l+r$ $\rightarrow$ 1 removed for root

We can choose any l nodes from $n-1$ to put it in as left subtree. $= {}^{n-1}C_l$

After choosing l nodes, r nodes will be fixed i.e. $n-l-1$
 $\downarrow$
 root

For each l we select, we can have many heaps with l elements & for each of these we can have many heaps with r elements.

$\therefore F(n) = {}^{n-1}C_l \cdot F(l) \cdot F(r)$ $\rightarrow$ Apply same for left & right subtree
 $\downarrow$
 choose particular l nodes (for this r is fixed)

We also have to find l and r for a given n .

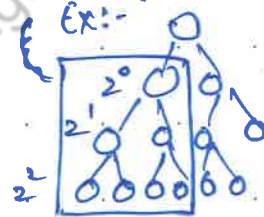
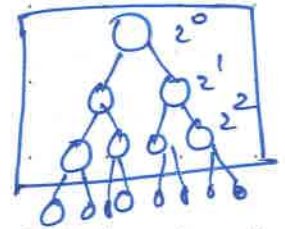
- height of heap $h = \log_2 n = h$
- Max. no. of nodes at last level $= 2^h = m$
- Elements actually present in last level of heap $= n - (2^h - 1) = p$

$\rightarrow$ No. of nodes present before last level.

Two cases of structures:-

- ① When last level is more than or equal to half-filled

$$2^0 + 2^1 + \dots + 2^{h-1} = \frac{1(2^h - 1)}{2 - 1} = 2^h - 1$$



$$(2^0 + 2^1 + \dots + 2^{h-1}) = 2^h - 1$$

That means left subtree is entirely filled up till the last level

and $l = 2^h - 1$ ~~where~~
 $r = n - l - 1$ ~~where~~
 (where $p \geq m/2$)

- ② When last level is less than half-filled:

$$l = 2^h - 1 - ((m/2) - p), \text{ if } p < m/2$$

That means right subtree's last level is completely empty.

- ⊛ We can also use DP for optimisation
 (Height of a complete binary tree $= \text{ceil}(\log_2(n+1)) - 1$)
 (Code in VScode)

Modular Multiplicative Inverse

Given two integers 'a' & 'm', find modular multiplicative inverse of 'a' under modulo 'm'. The modular multiplicative inverse is an integer 'x' such that,

$$ax \equiv 1 \pmod{m}$$

The value of x should be in $\{1, 2, \dots, m-1\}$ i.e. in the range of integer modulo m.

The multiplicative inverse of "a modulo m" exists if and only if a & m are relatively prime. (i.e. if $\gcd(a, m) = 1$)

I/P: - a = 3, m = 11
O/P: - 4

$\Rightarrow (4 \times 3) \% 11 = 1$,
where 4 is modulo inverse of 3 (under 11)

* 15 can also be valid but it doesn't come in $[1, 11]$ range.

(Works when m is prime)

$$a^{-1} = a^{m-2} \pmod{m}$$

Code

```
int inv(int a) {  
    int p = MOD-2;  
    int ans = 1;  
    while(p) {  
        if(p & 1) ans = (ans * a) % MOD;  
        a = a * a % MOD;  
        p = p / 2;  
    }  
    return ans;  
}
```

(Using fast exp. to find a^{m-2})

```
a = (a * a) % MOD;  
p >>= 1; }  
return ans; }
```

- The multiplicative inverse of a no. y is z iff $(z * y) == 1$

$$\frac{x}{y} = x * y^{-1} = \underline{x * z}$$

gives diff. values for each result

That's y we take $MOD = 10^9 + 7$ because its largest possible prime possible. helps to reduce overflow & result modulo inverse.

$\therefore$ WE WON'T PERFORM $\Rightarrow z = (x/y) \% M$
instead, WE PERFORM $\Rightarrow y_{inv} = \text{findMMI}(y, M);$
 $z = (x * y_{inv}) \% M;$

Calculating nCr

```
void factInit() {  
    int n = fact.size();  
    fact[0] = inv-fact[0] = 1;  
    for(int i = 1; i < n; i++) {  
        fact[i] = (fact[i-1] * i) % MOD;  
        inv-fact[i] = inv(fact[i]); }  
}
```

$$nCr = \frac{n!}{(n-r)! r!}$$

```
int ncr(int n, int r) {  
    return fact[n] * (inv-fact[n-r] * inv-fact[r] % MOD) % MOD;}
```


792. No. of Matching Subsequence (leetcode)

Given a string s , and an array of strings $words$, return the no. of $words[i]$ that is a subsequence of s .

I/P:- $s = "abcde"$

$words = ["a", "bb", "acd", "ace"]$

O/P:- 3

a, acd, ace are subseq of s.

Algo

each column has ~~char~~ index of next closest character ($ch - 'a'$)

String

	0	1	2	3	4	5	...	25
a	0							
b		1	2	3	4			
c			2	3	4			
d				3	4			
e					4			

(remaining posⁿ will be INT-MAX)

- From last row, we will start copy the ~~row~~ current row data to upper row.
- If ~~same~~ char occurs ~~with~~ whose posⁿ is already filled up then we over write.

Code

```
int dp[50005][26];
int numMatchingSubseq(string s, vector<string> & words) {
    int n = s.size();
    int m = words.size();
    if (n == 0) return 0;
    str ← fill(dp[n-1], dp[n-1] + 26, INT-MAX);
```

```
int ans = 0;
dp[n-1][s[n-1] - 'a'] = n-1; // last line allotment
for (int i = n-2; i >= 0; i--) { // last line running loop from end row by row we update new occurring elements
    int idx = s[i] - 'a';
    str ← Copy(dp[i+1], dp[i+1] + 26, dp[i]);
    dp[i][idx] = i;
```

```
3
for (int i = 0; i < m; i++) {
    string curr = words[i];
    int curr_row = 0;
    int j = 0;
    while (j < curr.size() && curr_row < n) {
        int idx = curr[j] - 'a';
        if (dp[curr_row][idx] == INT-MAX) break;
        curr_row = dp[curr_row][idx] + 1;
        j++;
    }
    if (j == curr.size()) // if we have traversed all through the string then that means it is subsequence
        ans++;
}
return ans;
```

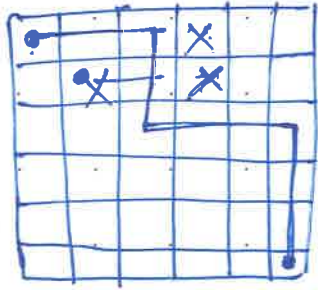
Your body holds deep wisdom. Trust in it. Learn from it. Nourish it. Watch your life transform and be healthy."

Max sum subarray

Q. You are given a matrix with some point as obstacles and we need to find the path as top as possible to reach (n, m) from $(0, 0)$.

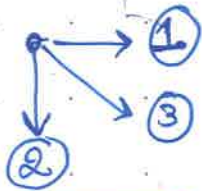
-1 → obstacles

0 → valid point



Top most path to reach $n \times m$. (shortest path which is lexico graphically smallest)

Algo



We set each neighbour point according to our priority i.e. we prioritize going $(0,1)$ over others. in case $(0,1)$ is not giving us valid path then we consider moving in $(1,0)$ and then $(1,1)$ which will add

Use of Transition State dp

* NOTE:-

this kinda

Q is done using another dp which tells us the state number (option) we need to take to get particular kind of path having multiple valid neighbours to go to.

to both coordinates leading to largest lexicographical path over other.

Code

```
void helper(int r, int c, vector<vector<int>>& matrix) {
```

```
    for (int i = r-1; i >= 0; i--) {
```

```
        for (int j = c-1; j >= 0; j--) {
```

```
            if (matrix[i][j] == 0) {
```

```
                dp[i][j] = 0;
```

```
                dp2[i][j] = -1;
```

```
                continue; }
```

```
            if (i == r-1 && j == c-1) {
```

```
                dp1[i][j] = 1;
```

```
                dp2[i][j] = 2; // Doesn't matter
```

```
                continue; }
```

```
            dp1[i][j] = 0;
```

```
            dp2[i][j] = -1;
```

```
            if (j != c-1 && dp2[i][j+1] != -1) {
```

```
                dp1[i][j] = (dp1[i][j] + dp1[i][j+1]) % MOD;
```

```
                dp2[i][j] = 1; }
```

```
            if (i != r-1 && dp2[i+1][j] != -1) {
```

```
                dp1[i][j] = (dp1[i][j] + dp1[i+1][j]) % MOD;
```

```
                dp2[i][j] = 2; }
```

```
            if (i != r-1 && j != c-1 && dp2[i+1][j+1] != -1) {
```

```
                dp1[i][j] = (dp1[i][j] + dp1[i+1][j+1]) % MOD;
```

```
                dp2[i][j] = 3; } }
```



```

cout << "No. of ways" << dp1[i][j] << endl;
if (dp2[0][0] == -1) {
    cout << "No. of path exists" << endl;
    return; }
cout << "Greatest path:";
int i=0, j=0;
while (i < r && j < c) {
    cout << "(" << i << ", " << j << ") ";
    if (dp2[i][j] == 1) { j++; }
    else if (dp2[i][j] == 2) { i++; }
    else { i++; j++; }
    cout << endl; }

```

Smaller to Larger merging

When we are given groups two different sizes to merge, we pop out all elements of group with lesser size & push it in bigger group. Doing this on many groups with various sizes will give us the merged group in $n \log n$ time. (where n = no. of total elements from all groups).

Reason :- When we merge smaller to larger, the new group has size $\geq 2a$ (where a = size of smaller grp, b = size of larger grp).

Since $b \geq a$
 when we merge the size of new group will be atleast double.
 That means grp size is getting increased in 1, 2, 4, 8 ... fashion.
 And thus each element only be shifted for $\log n$ time and for all elements $n \log n$. Thus $O(n \log n)$.

Trick :- If unordered_map ^{is gonna have} many elements then we can optimise it by reserving and predefining load-factor, after declaring umap.

```

umap.reserve(2048);
umap.max_load_factor(0.25);

```

(Resizing takes alot time & thus we reserve 2048 early.)

By default it is set to 1 (1 element per bucket in hash array)

Z Algorithm Pattern matching

$z(k)$ = longest substring starting at k which is also prefix of the string.

Ex:- aabxaayaab

0 1 2 3 4 5 6 7 8 9
z [0] [1] [0] [0] [2] [1] [0] [3] [1] [0]

(Calculating z array)

text = xabcaabzabc

pattern = abc

0 1 2 3 4 5 6 7 8 9 10 11 12 13
abc \$ x a b c a b z a b c

z 0 0 0 0 0 3 0 0 2 0 0 3 0 0

At this index

a == a 1 (At index 5 we store 3)
b == b 2
c == c 3
\$!= a stop

At index 5 and 11 we found the pattern.

Time: $O(n+m)$

Space: $O(n+m)$

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
a a b x a a b x c a a b x a a b x a y

z 0 1 0 0 4 1 0 0 0 8

After calculating 4th index z value we can use the remaining z values from previous z box. (We have to check if the len of word at that index touching (more than) ...)

if $z[i] = 3$ (Assumption)
then at index 5, $5+3 = 8$ but $z \text{ box}[4,7]$ so it is going out of the bound so we have to do further comparison. We can't just copy the z value from prev. z box directly.

0 1 0 0 4 1 0 0 0 8 1 0 0 4 5
When we check $13+4 = 17$ which is out of bound. So we have to do comparisons from this index. (13th index) (Previous posn has been filled up correctly)

So we again checking from arr[0]

arr[0] = arr[13] we shrink over z box
arr[1] = arr[14]
arr[2] = arr[15]
arr[3] = arr[16]
arr[4] = arr[17]

we found match for aabxa and thus we store 5 at index 13

0 1 0 0 4 1 0 0 0 8 1 0 0 5 1 0 0 1
Now we again start copying left z box values (and check the bounds too)

$17+4 = 21$ (out of bound)

So we start comparing for index 17th from starting.
 $\therefore \text{arr}[0] = \text{arr}[17] \rightarrow \text{len} = 1$
 $\text{arr}[1] \neq \text{arr}[18]$

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18

a a b x a a b x c a a b x a a b x a y

Z

0	1	0	0	4	1	0	0	0	8	1	0	0	5	1	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Code

```
vector<int> calcZ(vector<int> arr) {
    vector<int> z(arr.size());
    int left = 0;
    int right = 0;
    for (int k = 1; k < arr.size(); k++) {
        if (k > right) {
            left = right = k;
            while (right < arr.size() && arr[right] == arr[right-left])
                right++;
            z[k] = right - left;
            right--;
        } else {
            // we are operating inside box
            int kl = k - left;
            if (z[kl] < right - k + 1) {
                // if val doesn't stretch till right bound
                // then just copy it.
                z[k] = z[kl];
            } else {
                left = k;
                // otherwise try to see if there are more matches.
            }
        }
    }
}
```

while (right < arr.size() && arr[right] == arr[right-left])
 { right++; }
 $z[k] = \text{right} - \text{left};$
 right--;

}
 return z;

Rabin Karp Algorithm

Text $\rightarrow$ c c a c c a a e d b a (len = n)
 $3 \times 10^2 + 3 \times 10 + 1 \times 10^0$

pattern $\rightarrow$ d b a (len = m)
 hash = $4 \times 10^2 + 2 \times 10 + 1 \times 10^0$
 code = 421

$\alpha[d] \times 10^{m-1}$
 $+ \alpha[b] \times 10^{m-2}$
 $+ \alpha[a] \times 10^{m-3}$

- a - 1
- b - 2
- c - 3
- d - 4
- e - 5
- f - 6
- g - 7
- h - 8
- i - 9
- j - 10

we use sliding window and find out hash code if all alphabets if we get our substring then 26. we print.
 Avg $\rightarrow O(n-m+1)$
 Worst $\rightarrow O(mn)$ [when collision happens]

"Your prayer must be for a sound mind in a sound body."

Fenwick Tree / Binary Indexed Tree

The Range Sum Query and update can be solved using prefix sum.

Query $\rightarrow O(1)$

Update $\rightarrow O(n)$

Using Fenwick Tree we can have

Query $\rightarrow O(\log n)$

Update $\rightarrow O(\log n)$

(1-based indexing)

- Organized by lowest one bit (range of responsibility)

10000	1
1111	1
1110	1
1101	1
1100	1
1011	1
1010	1
1001	1
1000	1
0111	1
0110	1
0101	1
0100	1
0011	1
0010	1
0001	1

for ex:-

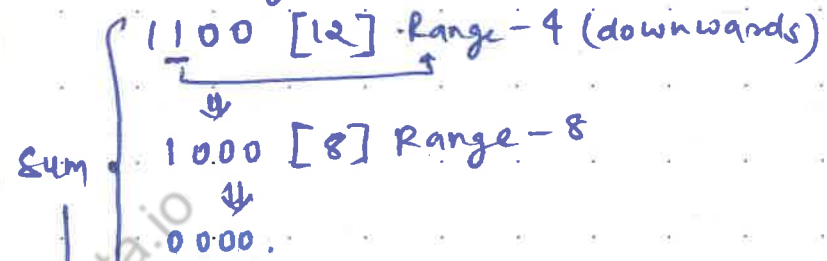
2 $\rightarrow$ 12 Range sum

[Val. at 1100 index
+ val. at 1000 index]

- Val. at 0001 index

prefix sums

- $\rightarrow$ start at current cell
- $\rightarrow$ staircase down until no bits are left (we will move by removing rightmost set bits which tells the range of that index towards downward)



will give us prefix sum from index 1 to 12.

(1-based indexing is followed)

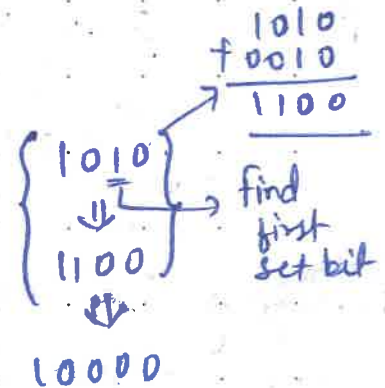
$\rightarrow$ loop runs at most total no. of bits.

$\rightarrow O(\log n) \rightarrow T.C$

Update

- $\rightarrow$ must update all cells that contains the updated cell in their range.

$\rightarrow$ We move up by adding lowest set bit to current index.



→ In worst case, all bit posⁿ are need to be added with

→ $O(\log n)$. → T.C

To find lowest ~~set~~ set bit

lowest set bit = $i \& (-i)$ or $i \& (n(i-1))$

Range query → $O(\log n)$

Range update → $O(n \log n)$ → for each element we

Point query → $O(\log n)$

~~Point~~ Point update → $O(\log n)$

will update those indexes which store these elements.

If we want to do range update and point query then we can use difference array.

Array → 5 3 7 9

Difference array → 5 -2 4 2
 ↓ first element (3-5) (9-7)
 remain same (7-3)

Point query $O(\log n)$

Take sum from index 0 to i . If we want to find original val at i .

Ex:- $arr[2] = 5 + (-2) + 4 = 7$

Range update ($O(\log n)$)

Array →

0	0	0	0	0
---	---	---	---	---

increase index 1 to 3 by x

0	x	x	x	0
---	---	---	---	---

Diff arr →

0	x	0	0	-x
	l		r	$r+1$

1's 10100
2's 01011
1
01100
10100
00100

∴ Just change

$diffarr[l] \pm x$

$diffarr[r+1] \mp x$

$l+1$ to r remains same in diff arr

arr → 5 3 7 9
 diff → 5 -2 4 2
 $l=1, r=2, inc \text{ by } 10$

arr' → 5 13 17 9
 diff → 5 8 4 -8
 $l \quad r \quad l+1$

$-2 + 10 = 8$

remains same ($l+1$ to r)

So we maintain difference array in fenwick tree instead of prefix sum to do the above.

Coordinate compression

When we solve fenwick tree problem we consider elements to be in range 0 to $n-1$ where n is a finite element less than 10^6 generally because we can't make array of size more than 10^6 so we need to compress values so that we can use fenwick tree.

Ex:- 2 4 2 3

We declare each element a rank:

2 4 2 3

0	2	0	1
---	---	---	---

→ New values will be map in 0 to $n-1$ (no. of elements)
in worst case

- we store elements in temp array
- use unique STL
- Then use lowerbound to access rank of particular element.

• Count sorted vowel strings (Leetcode)

Used P & C to solve in $O(1)$ → Trick

$$x_1 + x_2 + x_3 + x_4 + \dots + x_r = n \quad (x_i \geq 0)$$

then No. of combinations of x above x s such that

$$\sum_{i=1}^r x_i = n$$

$$\Rightarrow n + r - 1$$

Ex:- If we want to put sticks in n oranges (arranged in a line) to divide it into r groups

0 / 0 0 / 0 0

We require $r-1$ sticks. So we need to find $n-1$ positions to put these $r-1$ sticks.

$$x_1 + x_2 + \dots + x_r = n \quad (x_i \geq 0)$$

$$x_i = y_i - 1$$

$$y_1 + y_2 + \dots + y_r = n + r - 1 \quad \therefore \underline{n + r - 1}$$

"He who takes medicine and neglects to diet wastes the skill of his doctors."

Point updation

$$\begin{array}{r} 1010 \\ + 0010 \\ \hline 1100 \end{array} \rightarrow 12$$

↑ we move up

If we want to modify (add x), we check this index if it comes under some index's range.

Ex :- 12, 16 covers up to 10 index

Range sum Query

sum(2 to 6)
 $6 \rightarrow 0110$ will store sum from 6 to 5
 $4 \rightarrow 0100$ will store sum from 4 to 1 (inclusive)
 so that we extracted sum from 1 to 6
 same we have to do for 1 to 2.
 then subtract it to get sum from [3, 6]
 in $O(\log n)$

Point Query & Range update

arr $\rightarrow 5 \ 3 \ 7 \ 9$
 Diff $\rightarrow 5 \ -2 \ 4 \ 2$
 arr

to get value at i th index we need to find sum till i th from 0.

Ans.

in $O(\log n)$
 We will use difference array

If we want to update a range.

For example :- add 5 to all index from 2nd index initially, $x_1, x_2, x_3, x_4, \dots, x_{12}, x_{13}$ to 13th index.

$x_1, x_2 - x_1, x_3 - x_2, x_4 - x_3, \dots, x_{12} - x_{11}, x_{13} - x_{12}$

After adding 5.

$x_1, x_2 + 5, x_3 + 5, x_4 + 5, \dots, x_{12} + 5, x_{13} + 5, \dots$
 $x_1, x_2 - x_1, x_3 - x_2 + 5, x_4 - x_3, \dots, x_{12} - x_{11}, x_{13} - x_{12} - 5$

So we add 5 at 2nd index and subtract 5 at 13th index

In diff array, $[x_i + d, x_{j+1} - d]$
 for (i, j) be the range for which we adding 'd'.

Max Bipartite Matching

In a graph, how many connections/ or edges can be made b/w nodes such that we need to form bipartite graph and some nodes can be left coz one node can connect at most one node.

There are M job applicants and N jobs. Each applicant has a subset of jobs that he/she is interested in. Each job opening can only accept one applicant and a job applicant can be appointed for only one job. Find an assignment of jobs to applicants such that as many applicants as possible get jobs.

Leetcode #366

Algo.

- 1) We have an adj. matrix ~~also~~ $graph[M][N]$ where M applicants are interested in N jobs, and it is 1 where ' i ' applicant is interested in ' j ' job.
- 2) $match[N]$ arr will maintain i^{th} job is taken by which applicant. Initially set to -1 .
- 3) Then we run a loop thru all applicants to see if they can get a job.
- 4) We main $seen[N]$ for every new applicant.
- 5) ~~we~~ If an applicant can take a certain job then two possibilities are there :-
 - 1) The job is not taken
 - 2) The job is takenIn this case, we assign it & return true

↳ or else we check if assigned applicant can take any other jobs. If not, then we ~~return~~ check for other jobs for current applicant and if we still don't get one, we return false.

6) For each applicant we return true if he can get a job or else false.

Code

```
bool canGetJob (bool graph[M][N],
               int applicant, bool seen[],
               int match[]) {
    // Try every job one by one
    for (int v = 0; v < N; v++) {
        if (graph[applicant][v] &&
            !seen[v]) {
            seen[v] = true;
```

```
if (match[v] == -1 ||
    canGetJob (graph, match[v],
               seen, match)) {
```

```
    match[v] = u;
    return true;
```

```
    }
}
return false;
```

```
int maxMatch (bool graph[M][N]) {
```

```
    int match[N];
```

```
    memset (-1,
```

```
    int result = 0;
```

```
    for (int u = 0; u < M; u++) {
```

```
        bool seen[N];
```

```
        memset
```

```
        if (canGetJob (graph, u, seen, match))
```

```
            result++;
```

```
    }
    return result; }
```

Here, to maximise we as a current applicant ~~is~~ ~~asking~~ if finds a job that is ~~not~~ taken & we are interested in it then we can first ask the applicant who has taken this job to check for other jobs that he can take and if yes then we can just stop & return true or else we are still going to check other jobs ourselves.

Graph Imp things

- 1) Dijkstra - SSSP with undirected weighted graph
- 2) BFS - SSSP with { directed (weighted/unweighted) and undirected (unweighted) }
- 3) Undirected graph cycle detection - (prefer) DFS / BFS (or DSU)
- 4) Directed graph cycle detection - (DFS + visited) (for each branching maintain a visited unordered map to see which nodes are visited but mark them unvisited once we reach to the end of that branch while returning)
- 5) Dijkstra - can work for grid type problem as well where we move in weighted manner.
- 6) Floyd warshall - $i \rightarrow k \rightarrow j$
Find the city with the min no. of

neighbours at a Threshold distance.

7) Tarjan's Algo - Bridges (1992)

8) Kahn's Algo - (BFS Topo sort)

9) Kosaraju Algo - ~~also~~ No. of Strongly connected Components

10) Whenever applying DSU in a grid just convert the cells into node values ($row * m + col$)

11) BFS for the problems where cycles are there and it seems DP is best method for it.
(min multiplications to reach end)

12) Topological sort - shortest path in Directed Acyclic graph.
(Sequentially nodes are sorted so dijkstra is not required as it increases time complexity)